



Last Time

- Chomsky Hierarchy and Grammar
- Parsing Approaches for Context-Free Grammar (CFG)
 - Top-down Approach
 - Bottom-up Approach
- Parsing Algorithms
 - Shift-Reduce Parsing
 - CYK Parsing



Today's Class

- Probabilistic Context-Free Grammar (PCFG)
- Lexicalized PCFG
- Dependency Syntax
- Dependency Parsing
- Neural-based Dependency Parsing



Statistical Parsing

- Statistical parsing uses a probabilistic model of syntax in order to assign probabilities to each parse tree.
- Provides a principled approach to resolving syntactic ambiguity.
- Allows supervised learning of parsers from tree-banks of parse trees provided by human linguists.
- Also allows unsupervised learning of parsers from raw text, but the accuracy of such parsers has been limited.

Probabilistic Context Free Grammar (PCFG)



- A PCFG is a probabilistic version of a CFG where each production has a probability.
- Probabilities of all productions rewriting a given non-terminal must add to 1, defining a distribution for each non-terminal.
$$\sum_a P(A \rightarrow \alpha) = 1$$
- String generation is now probabilistic where production probabilities are used to non-deterministically select a production for rewriting a given non-terminal.



PCFG

-
- Assumption 1: Place invariance.
 - Assumption 2: Context-free.
 - Assumption 3: Ancestor-free.
 - The parse tree probability is the production of the probability of all applied parsing rules.



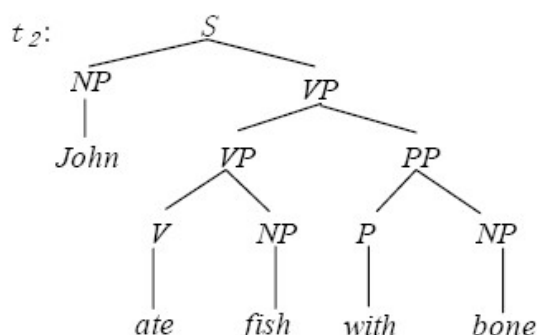
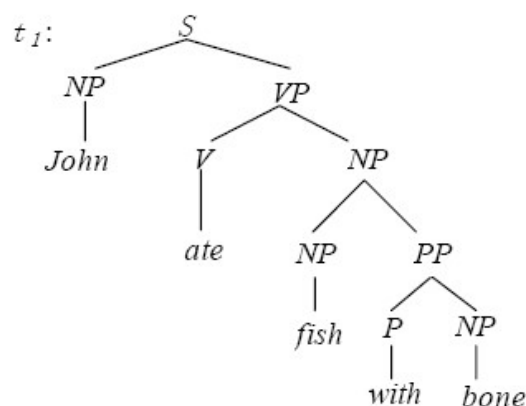
Simple PCFG Example

- Assume productions for each node are chosen independently.
- Probability of derivation is the product of the probabilities of its productions.
- Resolve ambiguity by picking most probable parse tree.



Simple PCFG Example

sentence = "John ate fish with bone"



$S \rightarrow NP VP$	1.0	$NP \rightarrow NP PP$	0.4
$PP \rightarrow P NP$	1.0	$NP \rightarrow John$	0.1
$VP \rightarrow V NP$	0.7	$NP \rightarrow bone$	0.18
$VP \rightarrow VP PP$	0.3	$NP \rightarrow star$	0.04
$P \rightarrow with$	1.0	$NP \rightarrow fish$	0.18
$V \rightarrow ate$	1.0	$NP \rightarrow telescope$	0.1

$$P(t_1) = 1.0 \times 0.1 \times 0.7 \times 1.0 \times 0.4 \times 0.18 \times 1.0 \times 1.0 \times 0.18 \\ = 0.0009072$$

$$P(t_2) = 1.0 \times 0.1 \times 0.3 \times 0.7 \times 1.0 \times 0.18 \times 1.0 \times 1.0 \times 0.18 \\ = 0.0006804$$

$$P(\text{sentence}) = P(t_1) + P(t_2) = 0.0015876$$



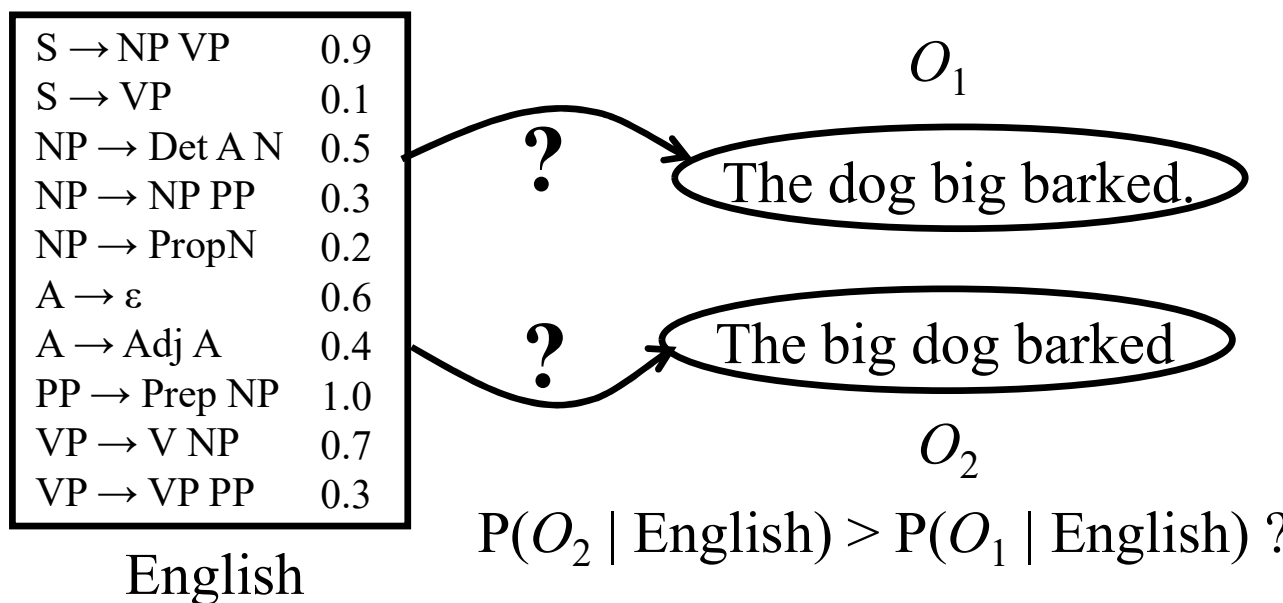
Three PCFG Tasks

- **Observation likelihood**: To classify and order sentences.
 $P(W|G)$ for sentence $W=w_1w_2...w_n$
 - Inside Algorithm
- **Most likely derivation**: To determine the most likely parse tree for a sentence.
 - Viterbi Algorithm
- **Maximum likelihood training**: To train a PCFG to fit empirical training data, i.e. maximize $P(W|G)$

PCFG: Observation Likelihood

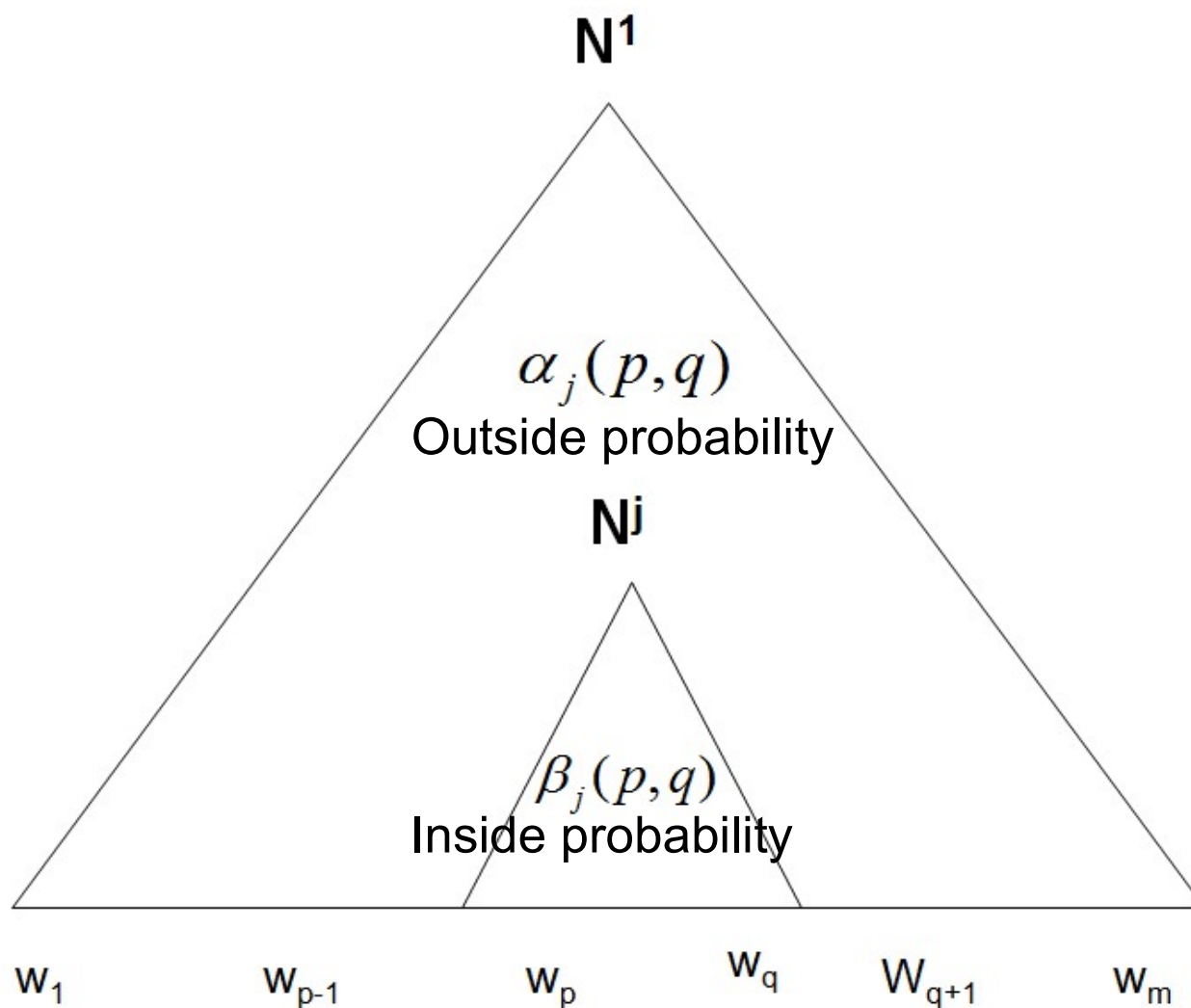


- There is an analog to Forward algorithm for HMMs called the **Inside algorithm** for efficiently determining how likely a string is to be produced by a PCFG.





Notation



Inside and Outside Probability



- Inside probability: total probability of generating words $w_p \dots w_q$ from non-terminal N^j .

$$\beta_j(p, q) = P(w_{pq} \mid N_{pq}^j)$$

- Outside probability: total probability of beginning with the start symbol N^1 and generating N_{pq}^j and all the words outside $w_p \dots w_q$

$$\alpha_j(p, q) = P(w_{1(p-1)}, N_{pq}^j, w_{(q+1)m})$$

- When $p > q$,

$$\alpha_j(p, q) = \beta_j(p, q) = 0$$



Probabilistic CKY Algorithm

- Revised CKY with probabilistic information.
- Set and maintain production probability during transfer grammar rule to CNF.
- Each cell provide a probability for each non-terminal.
- Cell[i,j]: the most likely parsing contains all of Non-terminals which covers Word_{i+1} to Word_j and its probability.



Probabilistic CKY Algorithm

Original Grammar

$S \rightarrow NP VP$	0.8
$S \rightarrow Aux NP VP$	0.1
$S \rightarrow VP$	0.1
$NP \rightarrow Pronoun$	0.2
$NP \rightarrow Proper-Noun$	0.2
$NP \rightarrow Det Nominal$	0.6
$Nominal \rightarrow Noun$	0.3
$Nominal \rightarrow Nominal Noun$	0.2
$Nominal \rightarrow Nominal PP$	0.5
$VP \rightarrow Verb$	0.2
$VP \rightarrow Verb NP$	0.5
$VP \rightarrow VP PP$	0.3
$PP \rightarrow Prep NP$	1.0

Chomsky Normal Form

$S \rightarrow NP VP$	0.8
$S \rightarrow X1 VP$	0.1
$X1 \rightarrow Aux NP$	1.0
$S \rightarrow book \mid include \mid prefer$ 0.01 0.004 0.006	
$S \rightarrow Verb NP$	0.05
$S \rightarrow VP PP$	0.03
$NP \rightarrow I \mid he \mid she \mid me$ 0.1 0.02 0.02 0.06	
$NP \rightarrow Houston \mid NWA$ 0.16 .04	
$NP \rightarrow Det Nominal$	0.6
$Nominal \rightarrow book \mid flight \mid meal \mid money$ 0.03 0.15 0.06 0.06	
$Nominal \rightarrow Nominal Noun$	0.2
$Nominal \rightarrow Nominal PP$	0.5
$VP \rightarrow book \mid include \mid prefer$ 0.1 0.04 0.06	
$VP \rightarrow Verb NP$	0.5
$VP \rightarrow VP PP$	0.3
$PP \rightarrow Prep NP$	1.0

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None			
	Det:.6	NP:.6*.6*.15 =.054		
		Nominal:.15 Noun:.5		

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 ← Nominal:.03 Noun:.1	None	VP:.5*.5*.054 =.0135		
	Det:.6	NP:.6*.6*.15 =.054		
		Nominal:.15 Noun:.5		

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:1, Verb:.5 Nominal:.03 Noun:.1	None	S: $.05 * .5 * .054 = .00135$ VP: $.5 * .5 * .054 = .0135$		
	Det:.6	NP: $.6 * .6 * .15 = .054$		
		Nominal:.15 Noun:.5		

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None	S :.05*.5*.054 =.00135 VP:.5*.5*.054 =.0135	None	
	Det:.6	NP:.6*.6*.15 =.054	None	
		Nominal:.15 Noun:.5	None	
			Prep:.2	

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None	S: $.05 * .5 * .054$ = .00135 VP: $.5 * .5 * .054$ = .0135	None	
	Det:.6	NP: $.6 * .6 * .15$ = .054	None	
		Nominal:.15 Noun:.5	None	
			Prep:.2	PP: $1.0 * .2 * .16$ = .032
				NP:.16 PropNoun:.8

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None	S:.05*.5*.054 =.00135 VP:.5*.5*.054 =.0135	None	
	Det:.6	NP:.6*.6*.15 =.054	None	
		Nominal:.15 Noun:.5	None	Nominal: .5*.15*.032 =.0024
			Prep:.2	PP:1.0*.2*.16 =.032
				NP:.16 PropNoun:.8

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None	S: $.05 * .5 * .054$ = .00135 VP: $.5 * .5 * .054$ = .0135	None	
	Det:.6 ←	NP: $.6 * .6 * .15$ = .054	None	NP: $.6 * .6 * .0024$ = .000864
		Nominal:.15 Noun:.5	None	Nominal: $.5 * .15 * .032$ = .0024
			Prep:.2	PP: $1.0 * .2 * .16$ = .032
				NP:.16 PropNoun:.8

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:. 5 Nominal:.03 Noun:.1		S:. $.05 * .5 * .054$ =.00135		S:. $.05 * .5 * .$.000864 =.0000216
	None	VP:. $.5 * .5 * .054$ =.0135	None	
				NP:. $.6 * .6 * .$.0024 =.000864
	Det:.6	NP:. $.6 * .6 * .15$ =.054	None	
		Nominal:.15 Noun:.5	None	Nominal: .5*.15*.032 =.0024
			Prep:.2	PP:1.0*.2*.16 =.032
				NP:.16 PropNoun:.8

Probabilistic CKY Algorithm



Book the flight through Houston

S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None	S:.05*.5*.054 =.00135 VP:.5*.5*.054 =.0135	None	S:.03*.0135* .032 =.00001296 S:.0000216
	Det:.6	NP:.6*.6*.15 =.054	None	NP:.6*.6* .0024 =.000864
		Nominal:.15 Noun:.5	None	Nominal: .5*.15*.032 =.0024
			Prep:.2	PP:1.0*.2*.16 =.032
				NP:.16 PropNoun:.8

Probabilistic CKY Algorithm



Book the flight through Houston

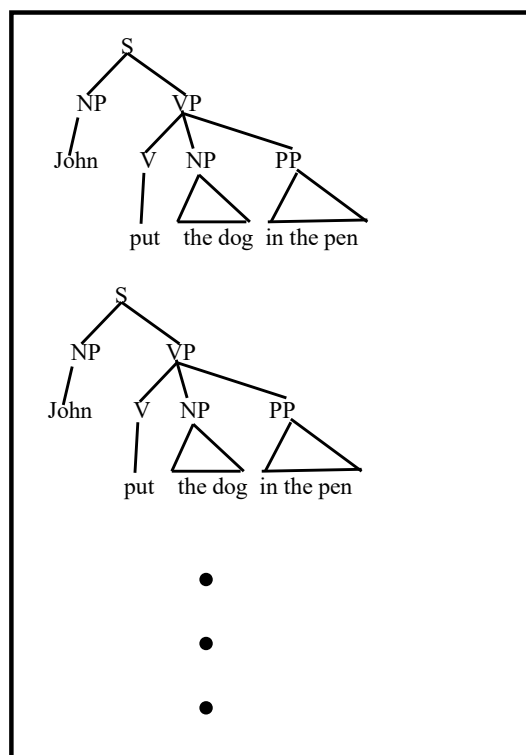
S :.01, VP:.1, Verb:. 3 Nominal:.03 Noun:.1		S:. 05 *.5*.054 =.00135		S:. 0000 216
	None	VP:. 5 *.5*.054 =.0135	None	
				NP:. 6 *.6* .0024 =.000864
	Det:. 6	NP:. 6 *.6*. 15 =.054	None	
				Nominal: .5*.15*.032 =.0024
		Nominal:. 15 Noun:. 5	None	
			Prep:. 2	PP:1.0*.2*.16 =.032
				NP:. 16 PropNoun:. 8



PCFG: Supervised Training

- If parse trees are provided for training sentences, a grammar and its parameters can be estimated directly from counts accumulated from the **tree-bank** (with appropriate smoothing).

Tree Bank



Supervised
PCFG
Training

$S \rightarrow NP VP$	0.9
$S \rightarrow VP$	0.1
$NP \rightarrow Det A N$	0.5
$NP \rightarrow NP PP$	0.3
$NP \rightarrow PropN$	0.2
$A \rightarrow \epsilon$	0.6
$A \rightarrow Adj A$	0.4
$PP \rightarrow Prep NP$	1.0
$VP \rightarrow V NP$	0.7
$VP \rightarrow VP PP$	0.3

English

Estimating Production Probabilities



- A set of production rules can be taken directly from the set of rewrites in the treebank.
- Parameters can be directly estimated from frequency counts in the treebank.

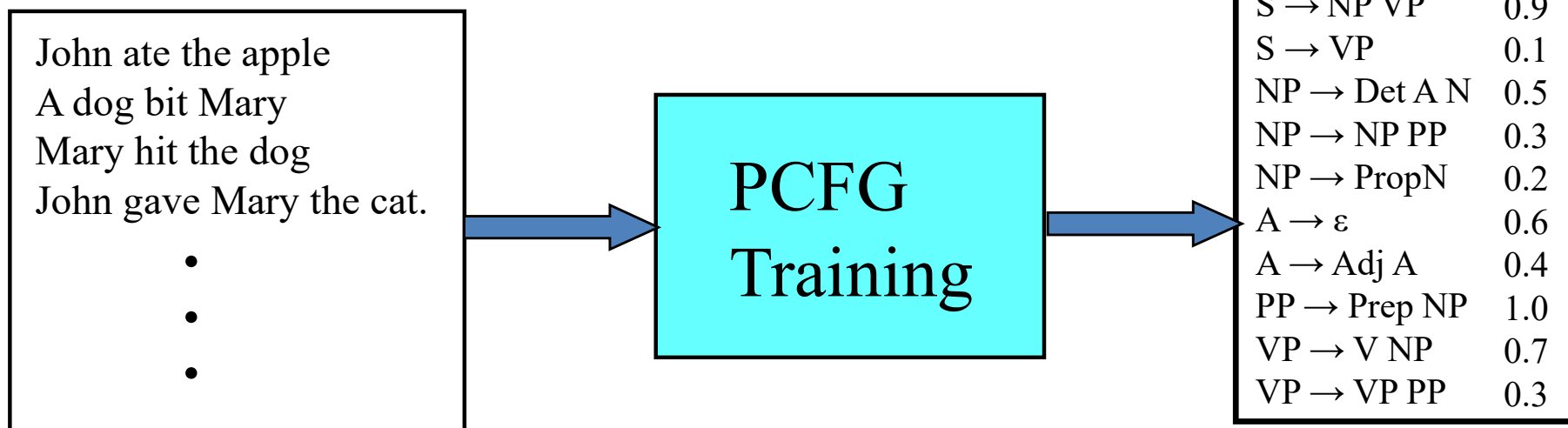
$$P(\alpha \rightarrow \beta \mid \alpha) = \frac{\text{count}(\alpha \rightarrow \beta)}{\sum_{\gamma} \text{count}(\alpha \rightarrow \gamma)} = \frac{\text{count}(\alpha \rightarrow \beta)}{\text{count}(\alpha)}$$

PCFG: Maximum Likelihood Training



- Given a set of sentences, induce a grammar that maximizes the probability that this data was generated from this grammar.
- Assume the number of non-terminals in the grammar is specified.
- Only need to have an unannotated set of sequences generated from the model. Does not need correct parse trees for these sentences. In this sense, it is **unsupervised**.

Training Sentences



English



Inside-Outside Algorithm

- The **Inside-Outside algorithm** is a version of EM for unsupervised learning of a PCFG.
 - Analogous to Baum-Welch (forward-backward) for HMMs.
- Given the number of non-terminals, construct all possible CNF productions with these non-terminals and observed terminal symbols.
- Use EM to iteratively train the probabilities of these productions to locally maximize the likelihood of the data.
- Experimental results are not impressive, but recent work imposes additional constraints to improve unsupervised grammar learning.



Summary of PCFG

- Pros
 - Disambiguation of structurally different parses.
 - Speed syntactic analysis by eliminating low probability structures.
 - Improve the robustness of analyzer by use of low probabilities.
 - Helpful to grammar induction.
- Cons
 - Since probabilities of productions do not rely on specific words or concepts, only general structural disambiguation is possible.
 - cannot resolve syntactic ambiguities that require semantics to resolve, e.g. ate with fork vs. meatballs.



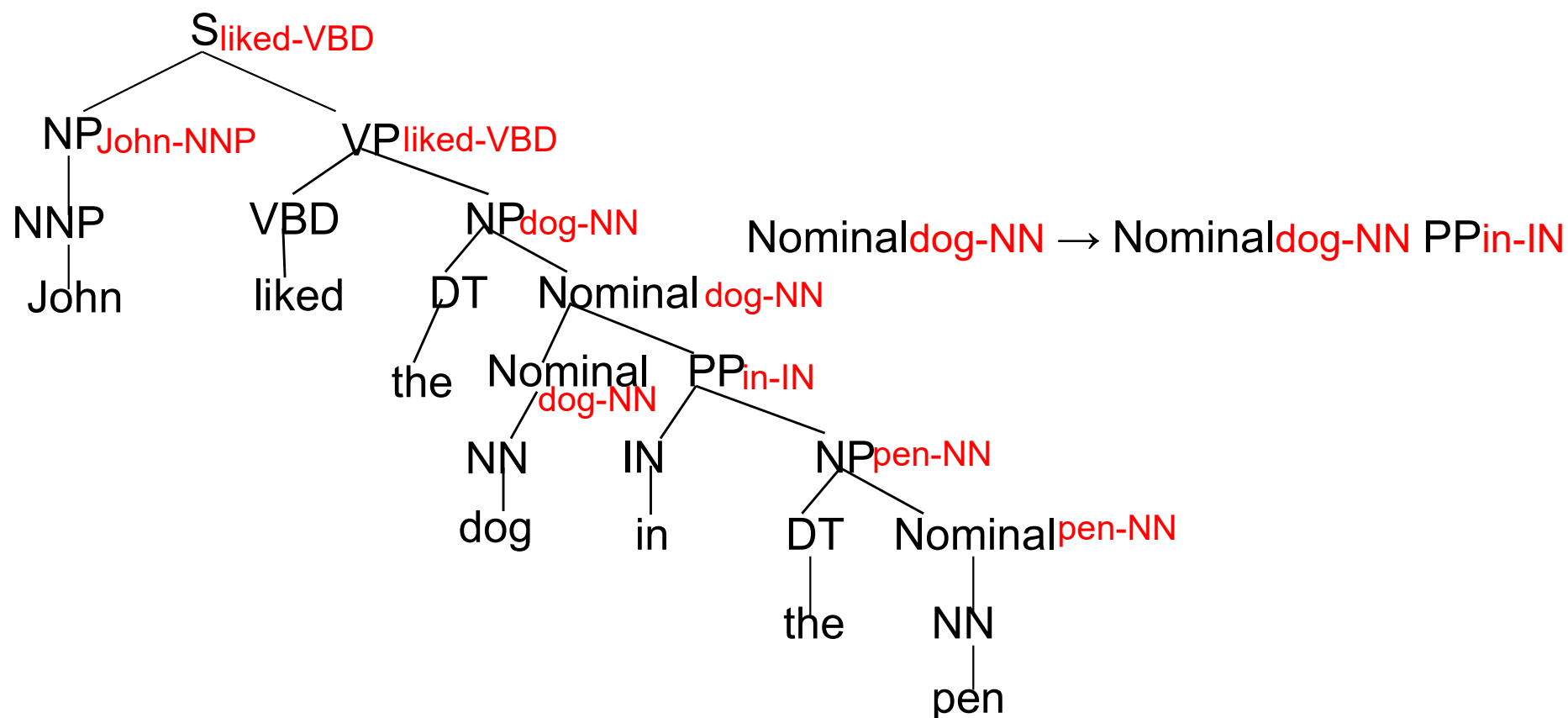
Lexicalized PCFG

- PCFGs must be **lexicalized**, i.e. productions must be specialized to specific words by including their head-word in their LHS non-terminals.
- Each **non-terminal** is associated to **headword** w .
- Syntactic phrases usually have a word in them that is most “central” to the phrase.
- Process Rule
 - Head of a VP is the main verb.
 - Head of an NP is the main noun.
 - Head of a PP is the preposition.
 - Head of a sentence is the head of its VP.



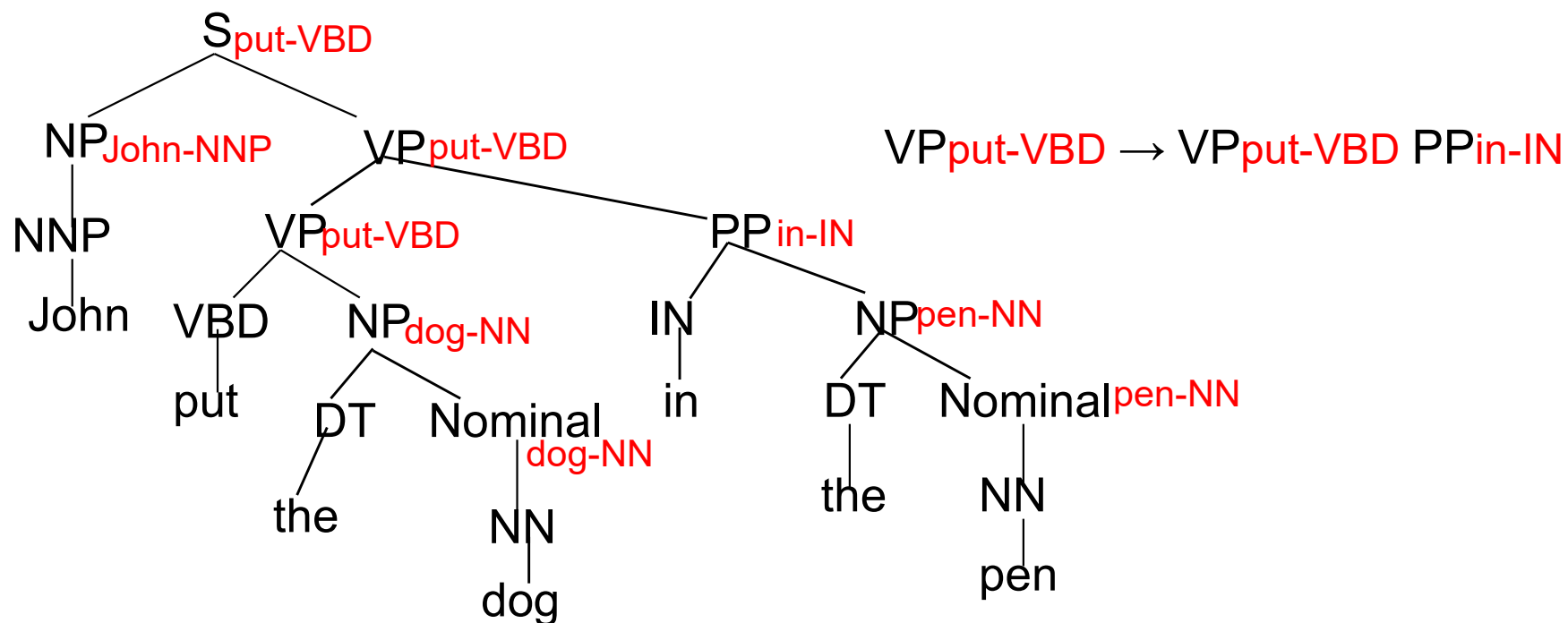
Lexicalized Productions

- Specialized productions can be generated by including the head word and its POS of each non-terminal as part of that non-terminal's symbol.





Lexicalized Productions



Parameterizing Lexicalized Productions



- Accurately estimating parameters on such a large number of very specialized productions requires enormous amounts of treebank data.
- Need some way of estimating parameters for lexicalized productions that makes reasonable independence assumptions so that accurate probabilities for very specific rules can be learned.



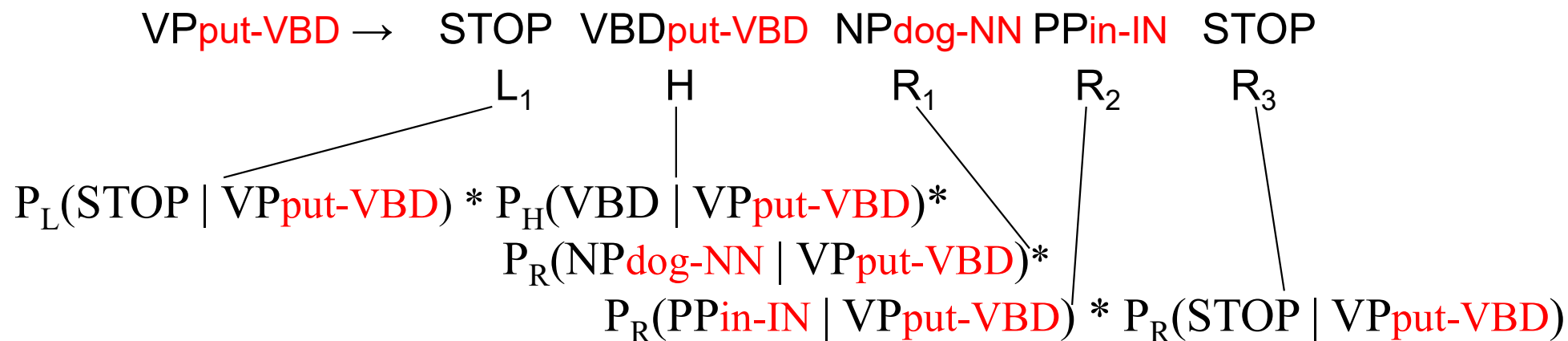
Collins' Parser

- Collins' (1999) parser assumes a simple generative model of lexicalized productions.
- Models productions based on context to the left and the right of the head daughter.
 - $LHS \rightarrow L_n L_{n-1} \dots L_1 H R_1 \dots R_{m-1} R_m$
- First generate the head (H) and then repeatedly generate left (L_i) and right (R_i) context symbols until the symbol STOP is generated.



Sample Production

$VP_{\text{put-VBD}} \rightarrow VBD_{\text{put-VBD}} NP_{\text{dog-NN}} PP_{\text{in-IN}}$



Estimating Production Generation Parameters



- Estimate P_H , P_L , and P_R parameters from treebank data.

$$P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{put-VBD}}) = \frac{\text{Count}(\text{PP}_{\text{in-IN}} \text{ right of head in a VP}_{\text{put-VBD}} \text{ production})}{\text{Count}(\text{symbol right of head in a VP}_{\text{put-VBD}})}$$

$$P_R(\text{NP}_{\text{dog-NN}} \mid \text{VP}_{\text{put-VBD}}) = \frac{\text{Count}(\text{NP}_{\text{dog-NN}} \text{ right of head in a VP}_{\text{put-VBD}} \text{ production})}{\text{Count}(\text{symbol right of head in a VP}_{\text{put-VBD}})}$$

- Smooth estimates by linearly interpolating with simpler models conditioned on just POS tag or no lexical info.

$$\begin{aligned} \text{sm}P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{put-VBD}}) &= \lambda_1 P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{put-VBD}}) \\ &\quad + (1 - \lambda_1) (\lambda_2 P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{VBD}}) + \\ &\quad \quad (1 - \lambda_2) P_R(\text{PP}_{\text{in-IN}} \mid \text{VP})) \end{aligned}$$



Parsing Evaluations Metrics

- PARSEVAL metrics measure the fraction of the constituents that match between the computed and human parse trees. If P is the system's parse tree and T is the human parse tree (the “gold standard”):
 - **Recall** = $(\# \text{ correct constituents in } P) / (\# \text{ constituents in } T)$
 - **Precision** = $(\# \text{ correct constituents in } P) / (\# \text{ constituents in } P)$
- **Labeled Precision** and **labeled recall** require getting the non-terminal label on the constituent node correct to count as correct.
- **F_1** is the harmonic mean of precision and recall.



Summary of Probabilistic Parsing

- Statistical models such as PCFGs allow for probabilistic resolution of ambiguities.
- PCFGs can be easily learned from treebanks.
- Lexicalization and non-terminal splitting are required to effectively resolve many ambiguities.
- Current statistical parsers are quite accurate but not yet at the level of human-expert agreement.



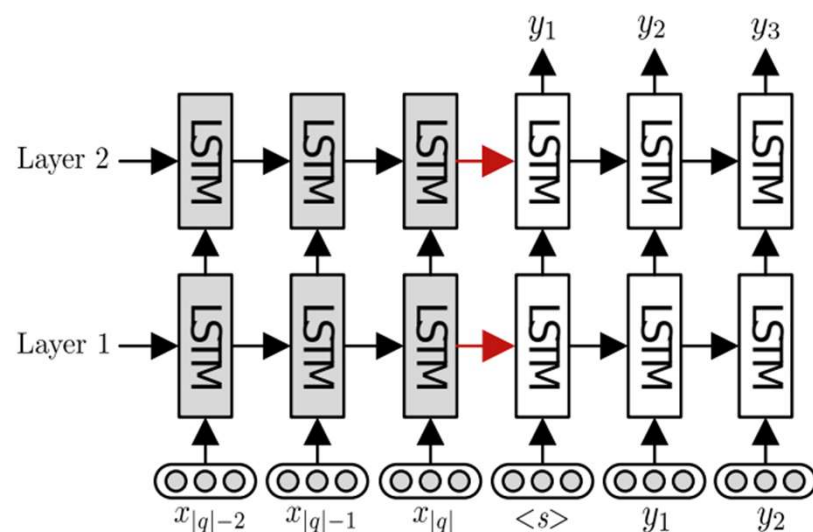
Neural-based Syntactic Summary

- Encoder
 - LSTM
 - Transformer
- Decoder
 - Transition-based
 - Chart-based
 - Sequence-to-Sequence



Neural-based Constituency Parsing

- [Li et al. 2016] present a general method based on an attention-enhanced encoder-decoder model. They encode input utterances into vector representations, and generate their logical forms by conditioning the output sequences or trees on the encoding vectors.





Neural-based Constituency Parsing

- Model Architecture

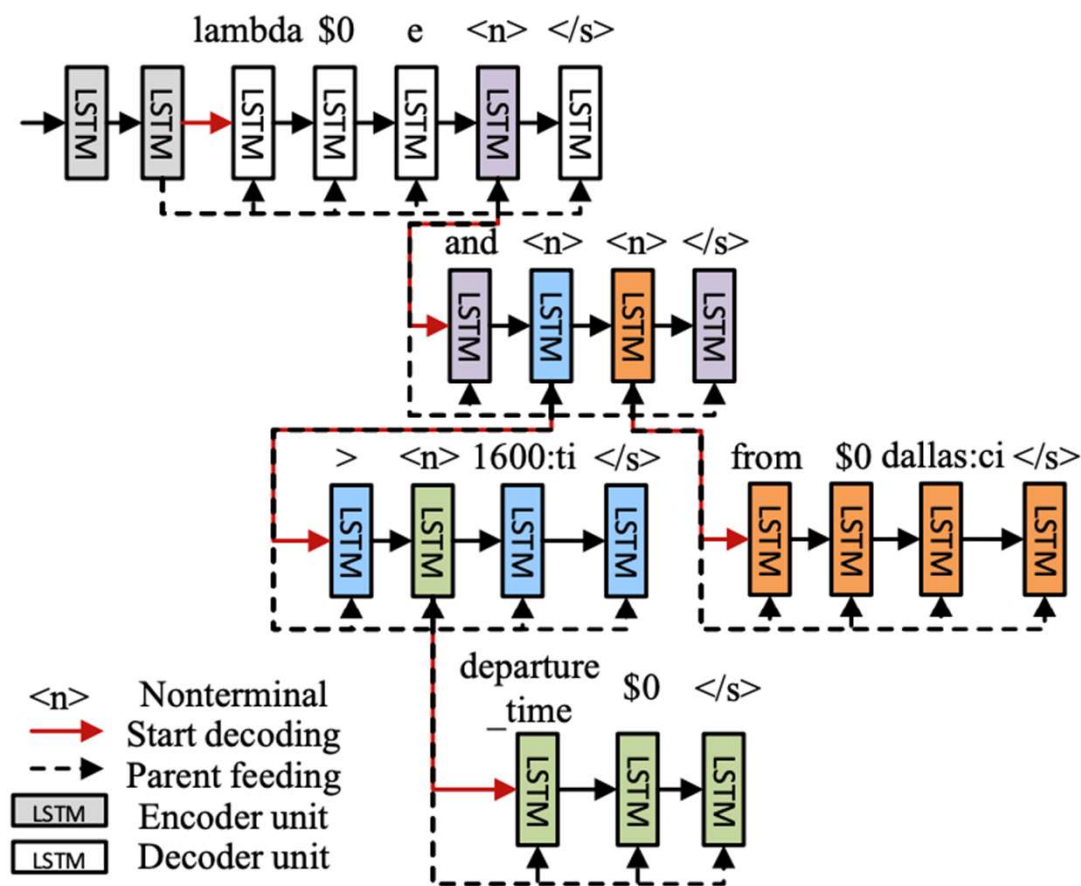


Figure 3: Sequence-to-tree (SEQ2TREE) model with a hierarchical tree decoder.



Neural-based Constituency Parsing

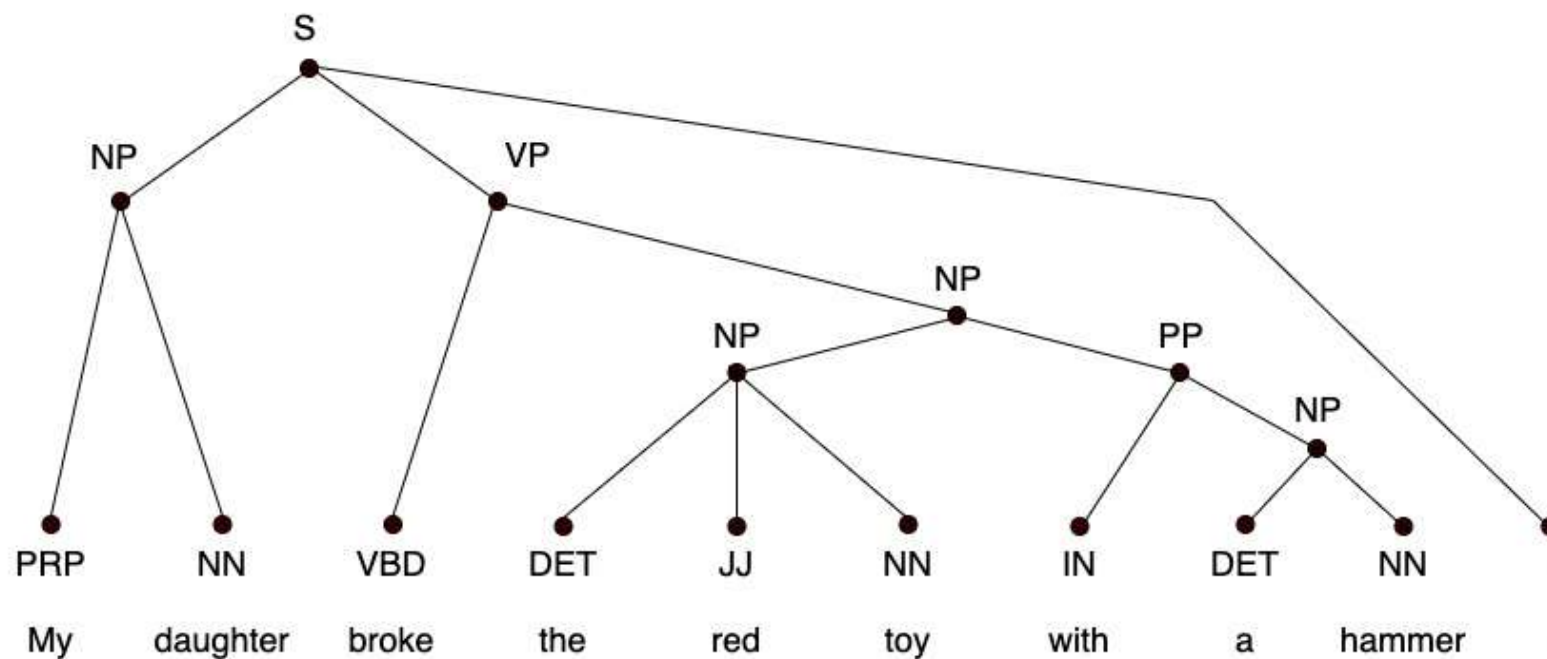
- [Gomez-Rodriguez et al. 2018] introduce a method to reduce constituent parsing to sequence labeling.
 - For each word w_t , it generates a label that encodes: (1) the number of ancestors in the tree that the words w_t and w_{t+1} have in common (CA), and (2) the non-terminal symbol at the lowest common ancestor (LCA).
 - They first prove that the proposed encoding function is injective for any tree without unary branches.
 - In practice, the approach is made extensible to all constituency trees by collapsing unary branches.

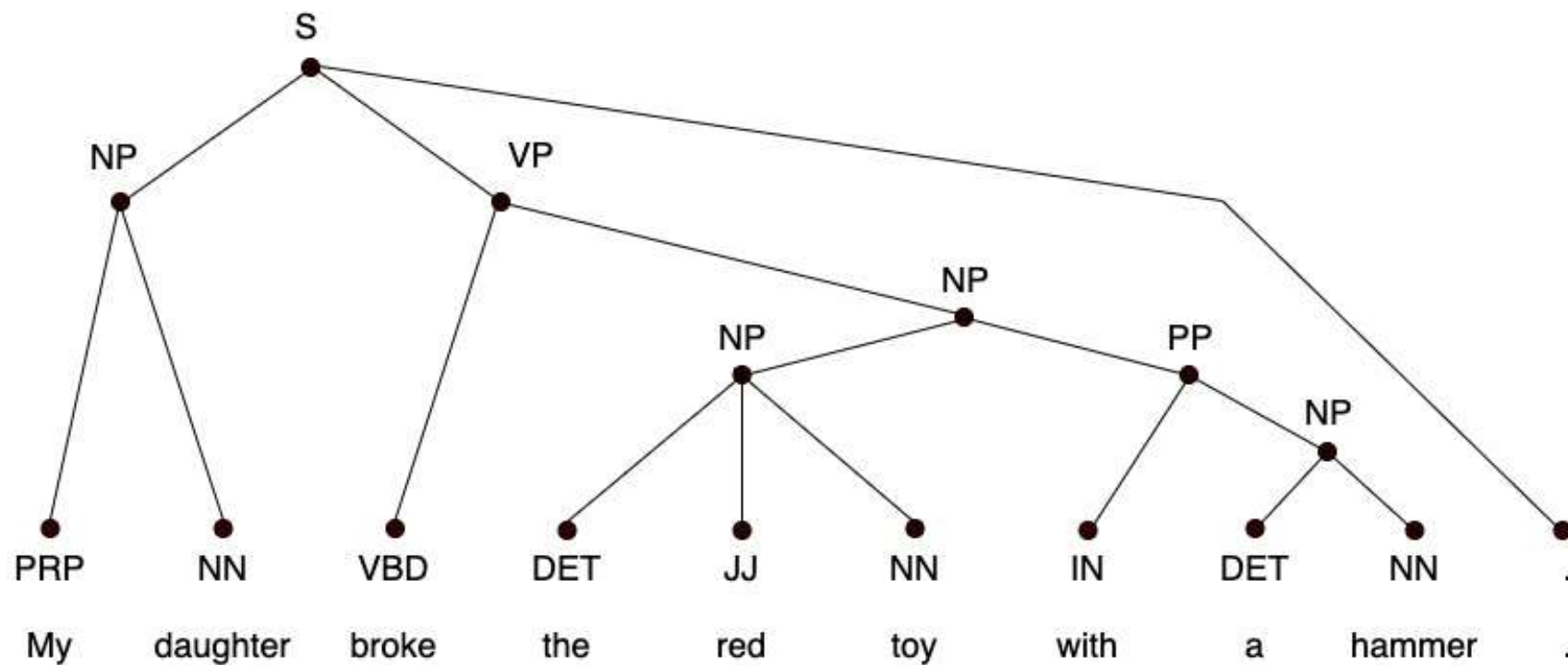
Example:

Parsing as Sequence Labeling - 1



- Example of the parsing tree



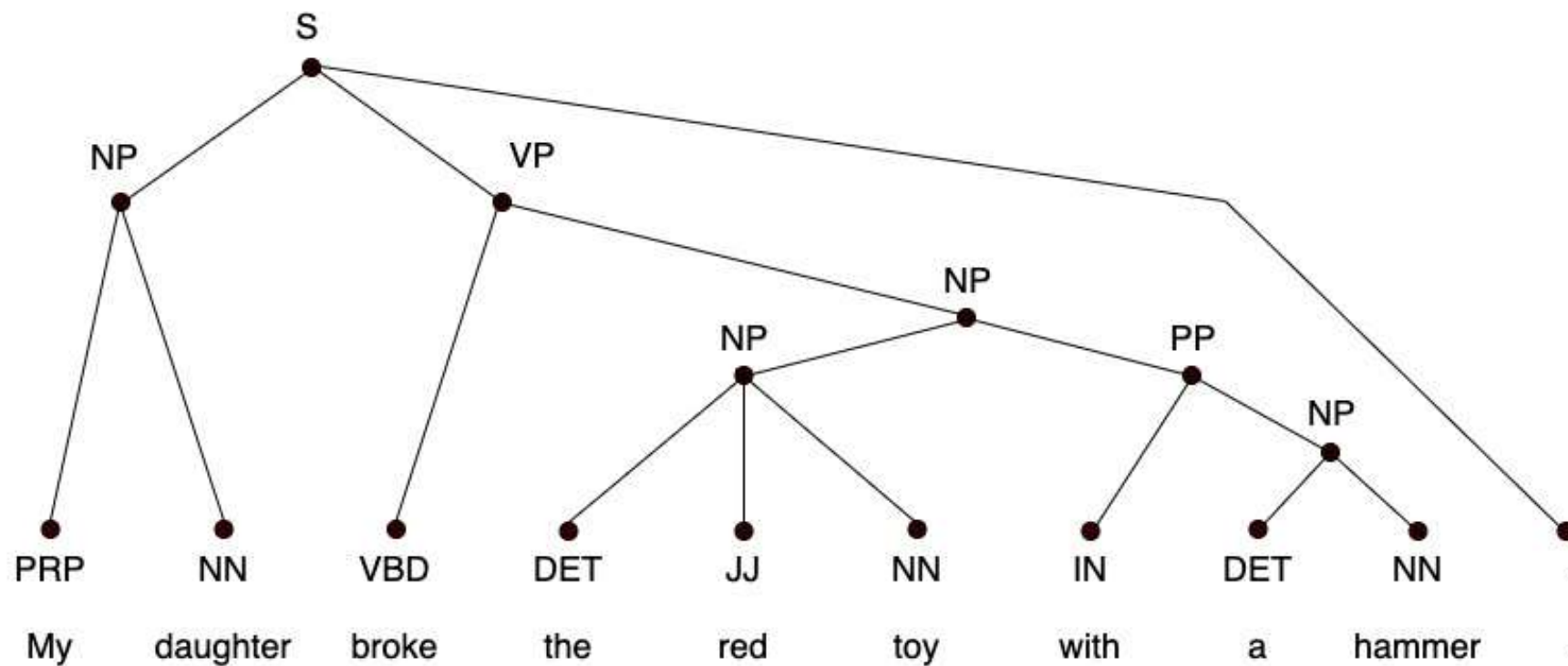


Linearized tree (absolute scale):

2NP

Linearized tree (relative scale):

2NP

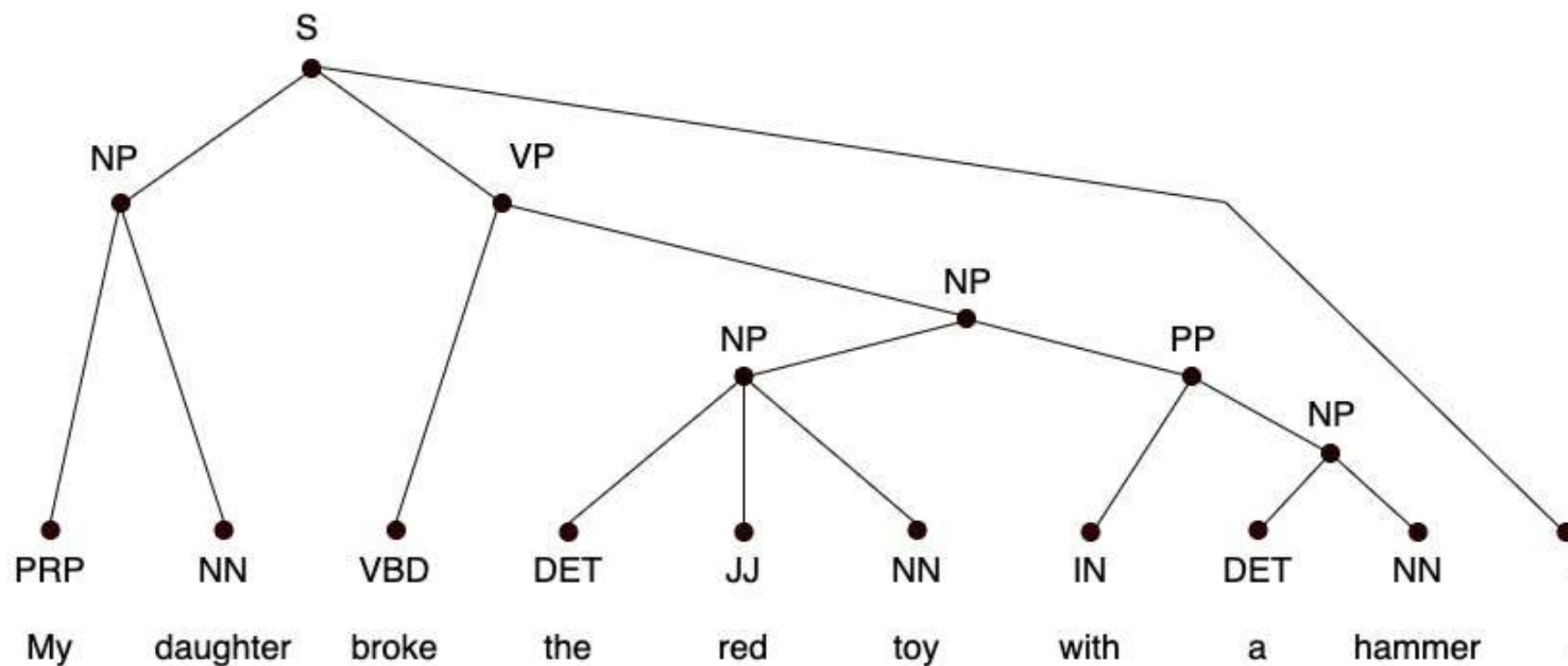


Linearized tree (absolute scale):

2NP 1S

Linearized tree (relative scale):

2NP -1S

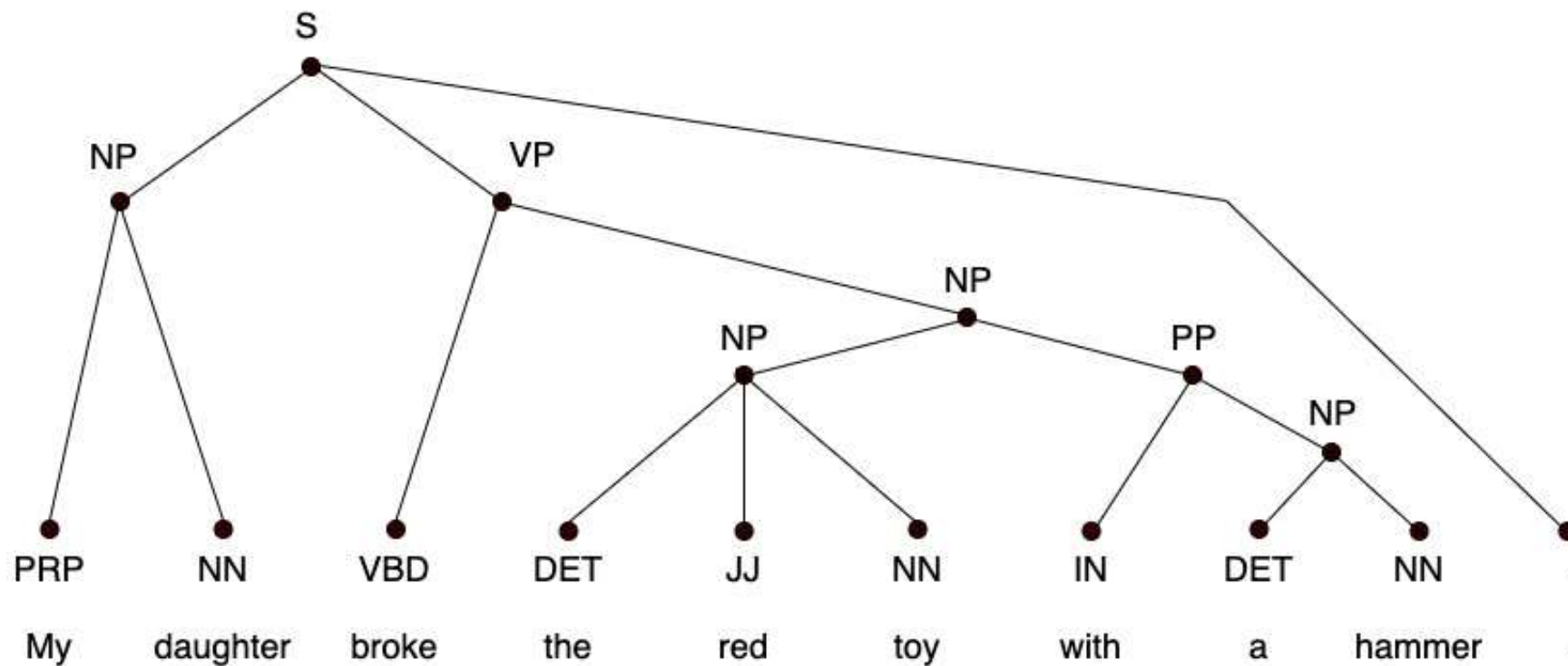


Linearized tree (absolute scale):

2NP 1S 2VP

Linearized tree (relative scale):

2NP -1S 1VP

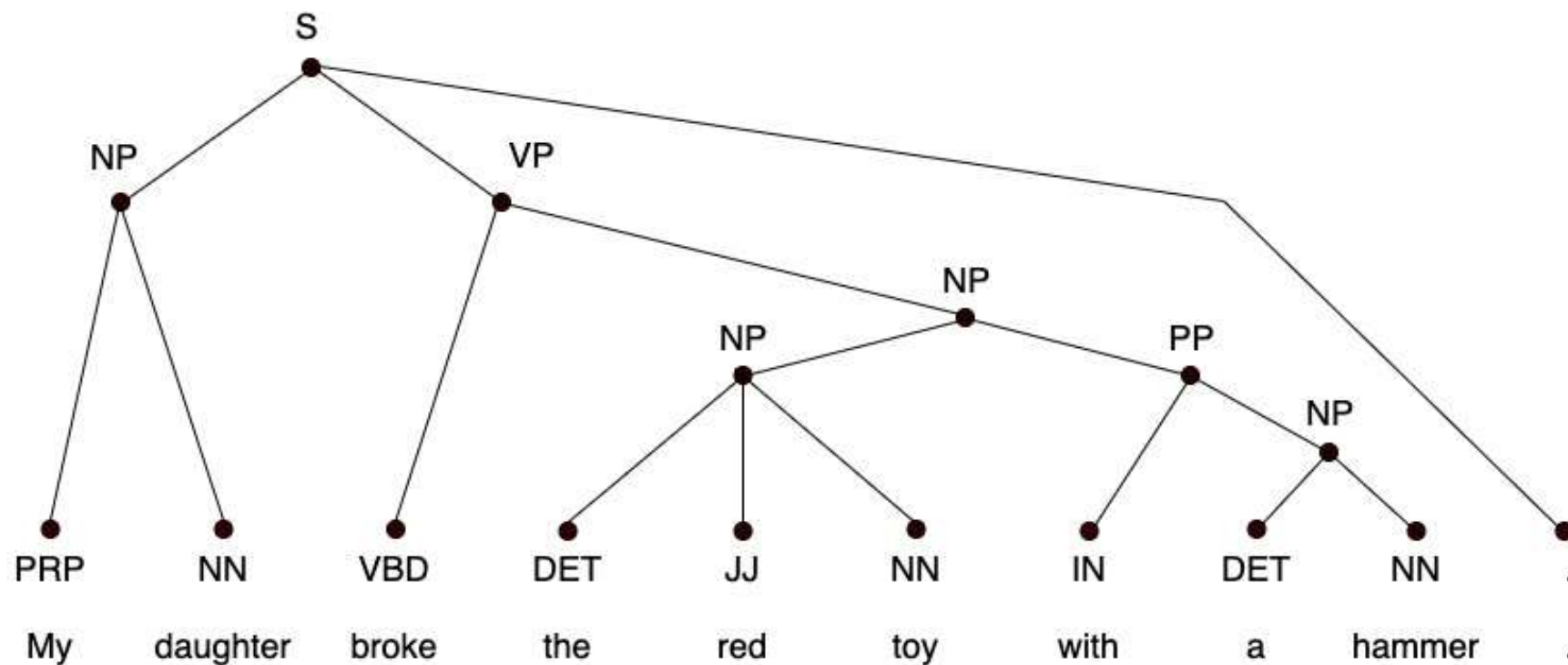


Linearized tree (absolute scale):

2NP 1S 2VP 4NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP

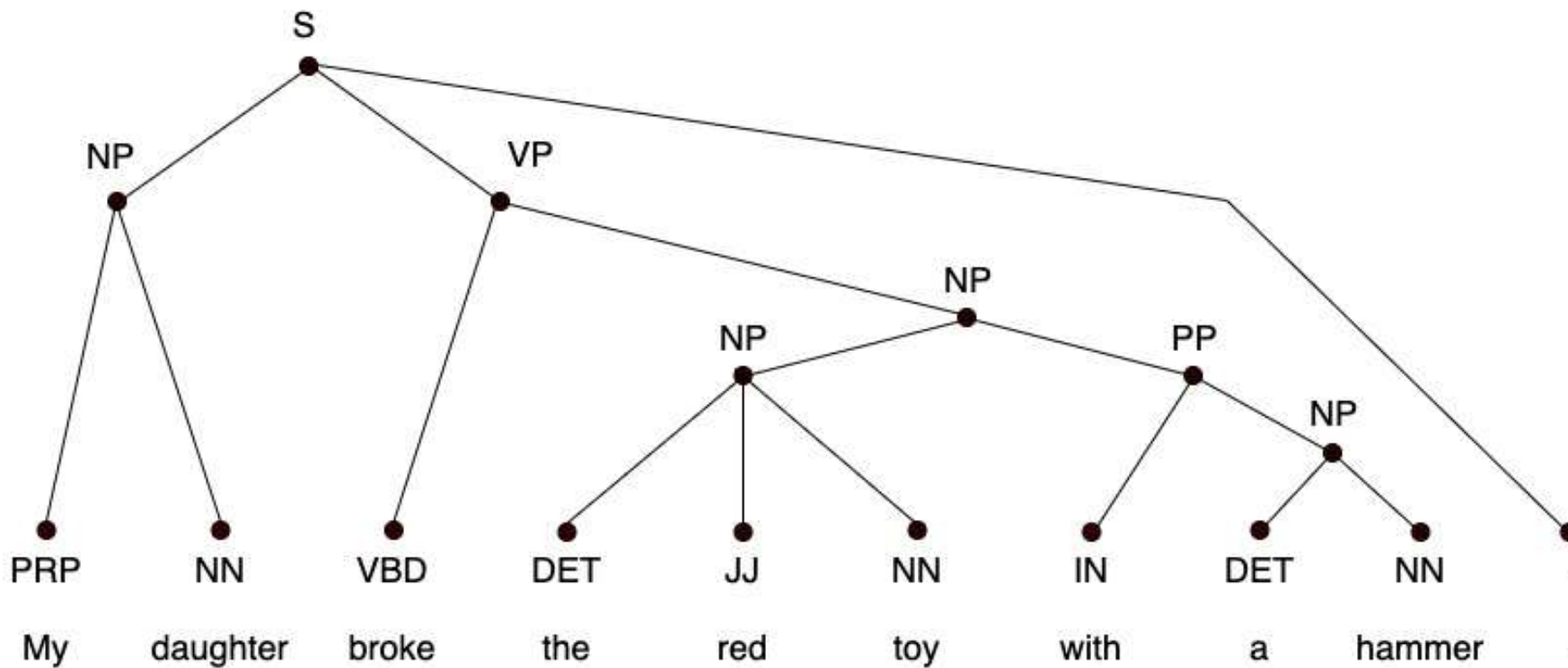


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP

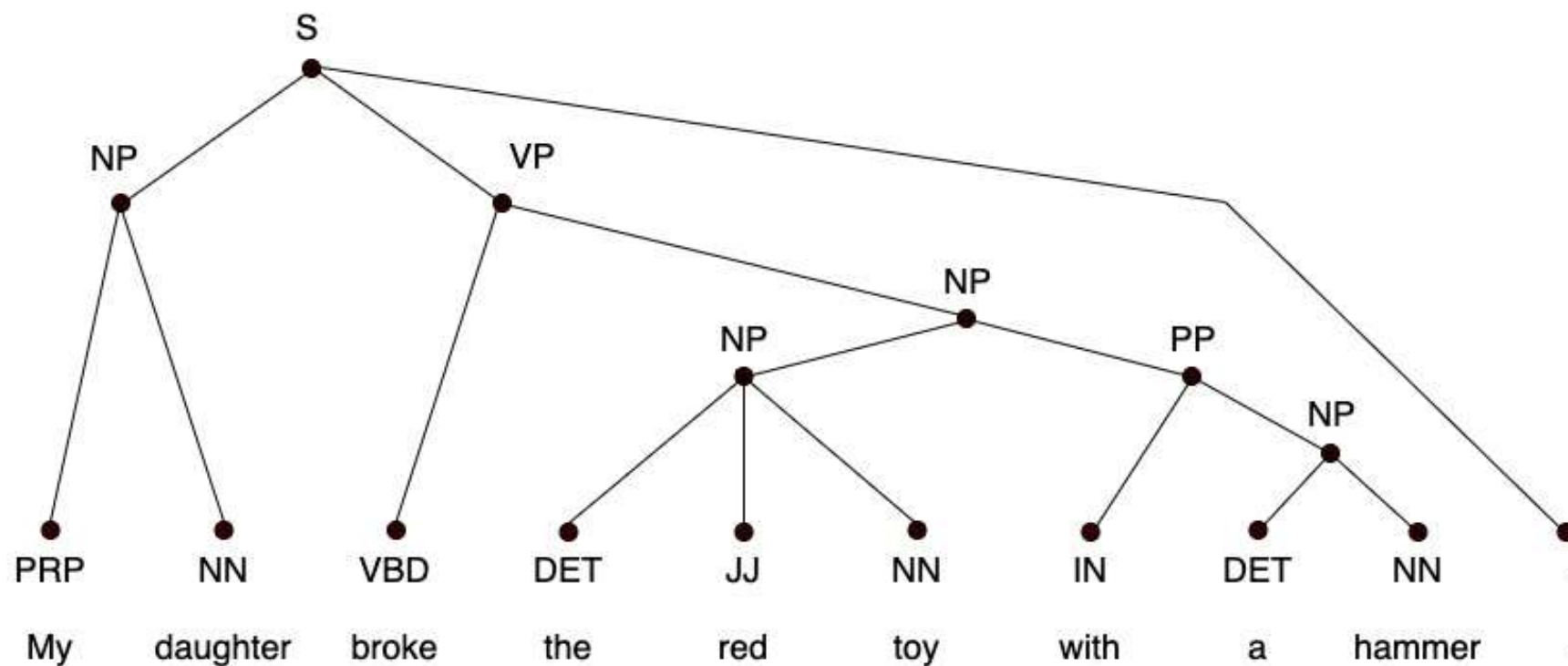


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP

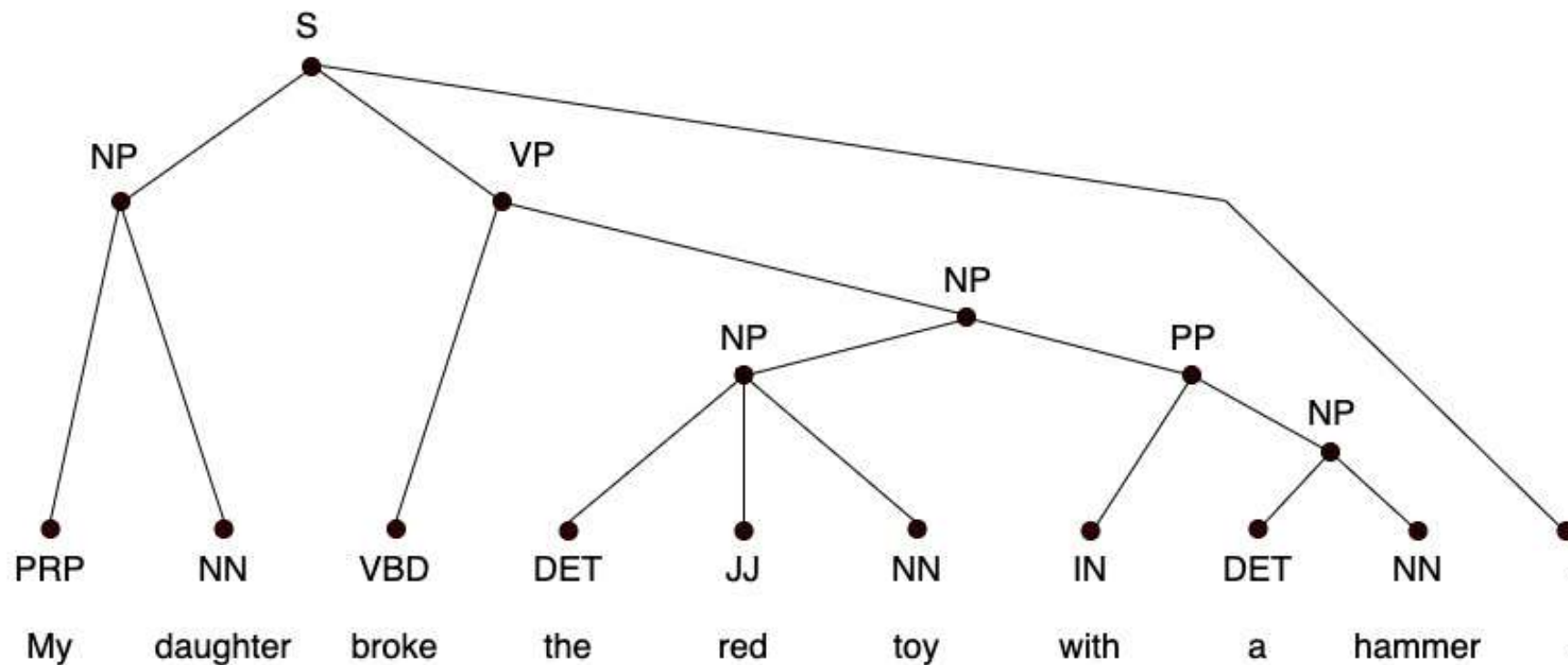


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP 4PP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP 1PP

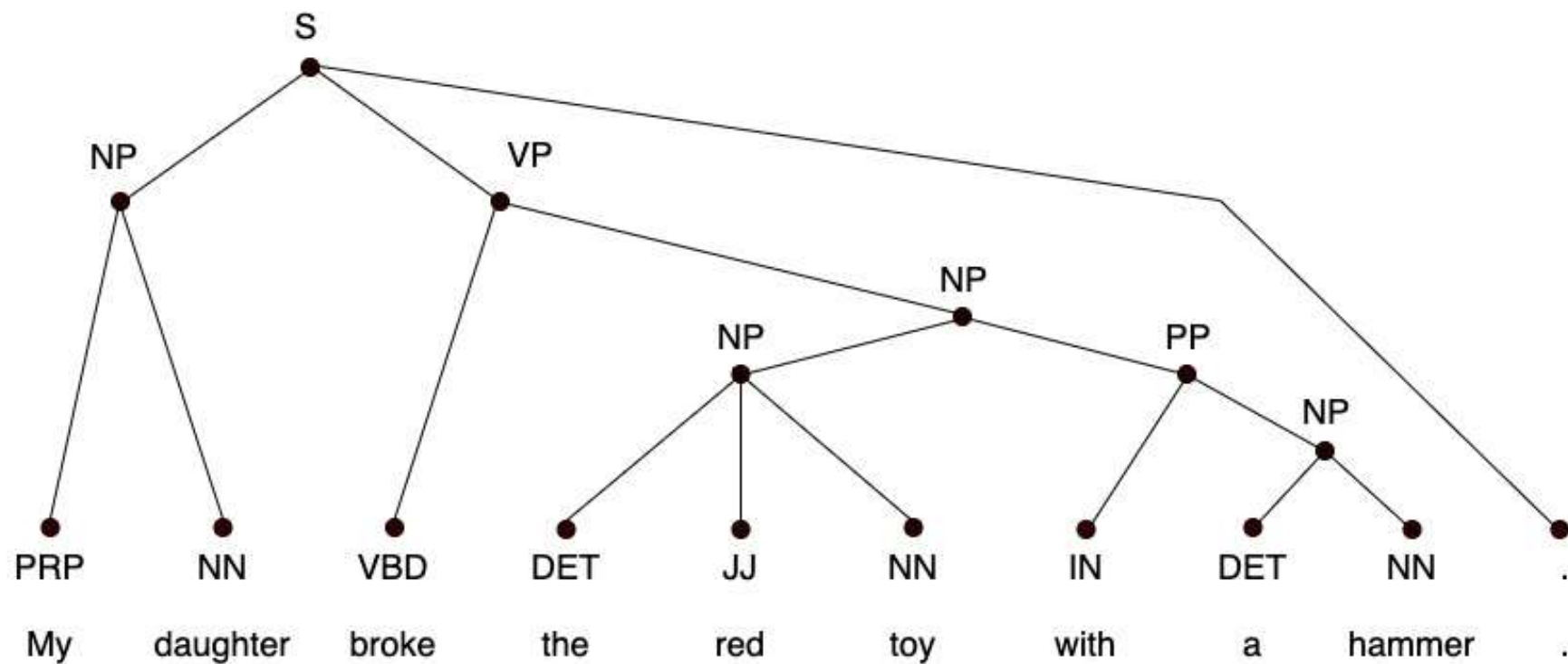


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP 4PP 5NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP 1PP 1NP



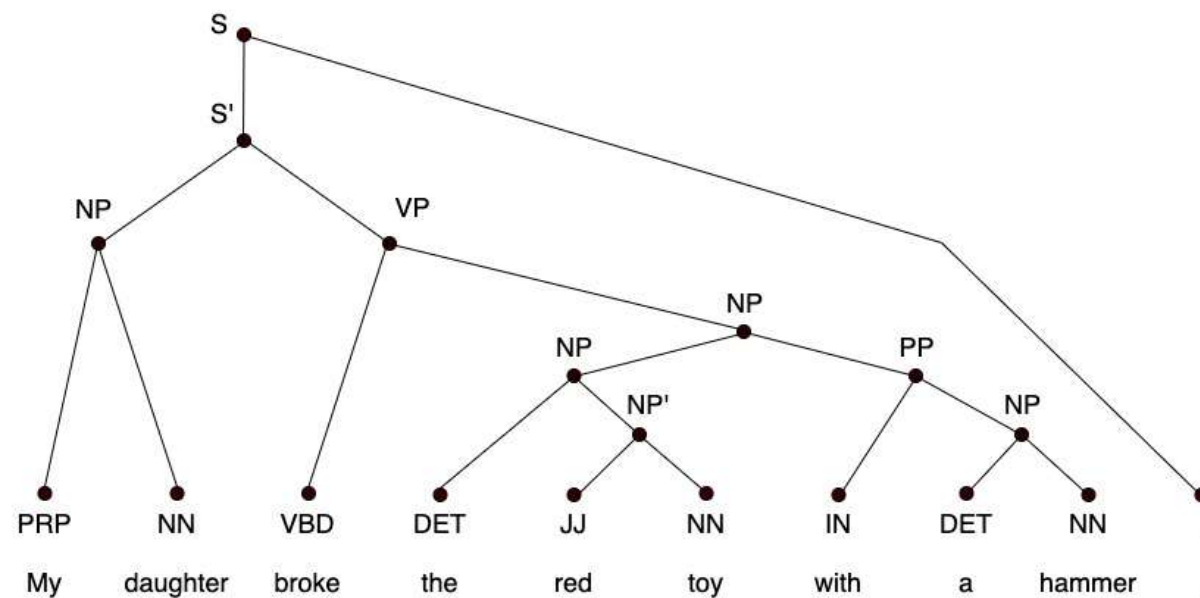
Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP 4PP 5NP 1S

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP 1PP 1NP -4S

- Example of the 2-ary parsing tree



Linearized tree (absolute scale):

3NP 2S' 3VP 5NP 6NP' 4NP 5PP 6NP 1S

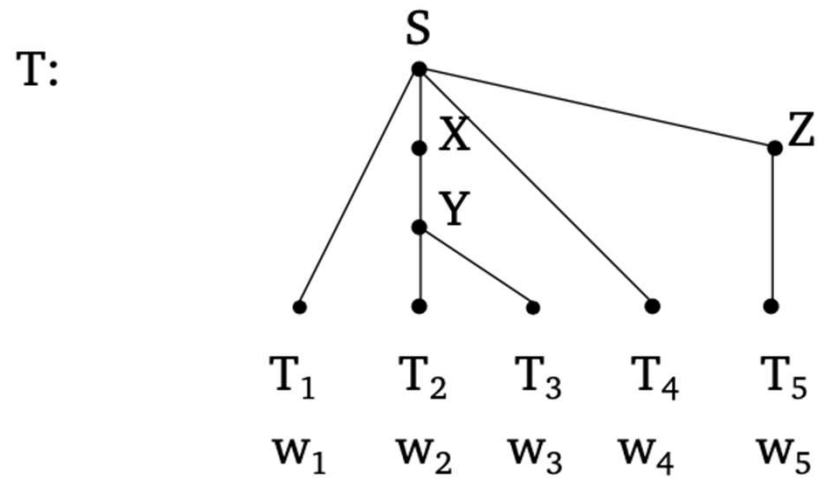
Linearized tree (relative scale):

3NP -1S' 1VP 2NP 1NP' -2NP 1PP 1NP -5S

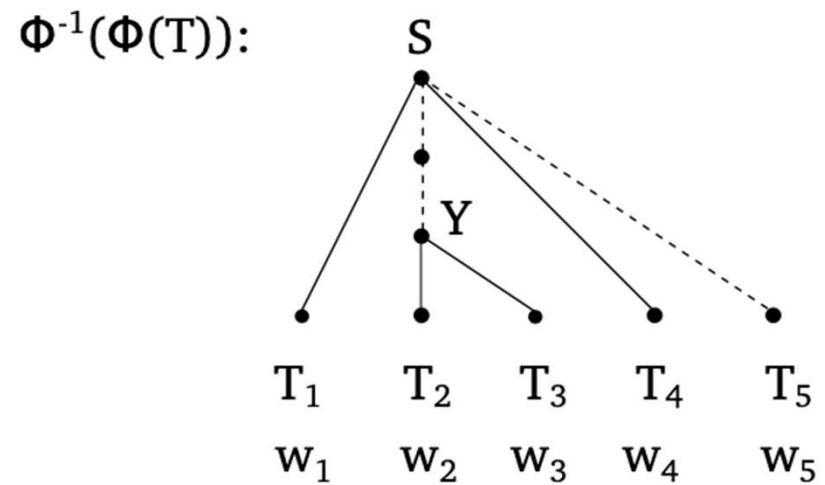
Linearized tree (simplified relative scale):

3NP -S' 1VP 2NP 1NP' -NP 1PP 1NP -S

- Example of the tree can not be directly linearized



$\Phi(T)$: 1_S 3_Y 1_S 1_S





Today's Class

- Neural-based Syntactic Parsing
- **Dependency Syntax**
- Dependency Parsing
- Neural-based Dependency Parsing
- Parsing Evaluation
- Parsing Resources



Dependency

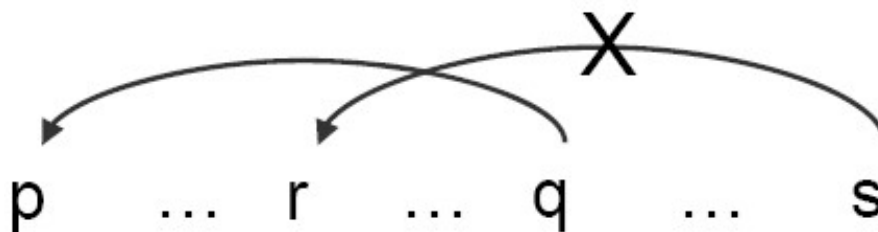
- Syntactic structure consists of **lexical items**, linked by binary asymmetric relations called **dependencies**.
- [Tesniere 1959]

The sentence is an **organized whole**, the constituent elements of which are **words**. Every word that belongs to a sentence ceases by itself to be isolated as in the dictionary. Between the word and its neighbors, the mind perceives **connections**, the totality of which forms the structure of the sentence. The structural connections establish **dependency** relations between the words. Each connection in principle unites a **superior** term and an **inferior** term. The superior term receives the name **governor**. The inferior term receives the name **subordinate**.



Robinson's axiom

- a. One and only one element is **independent**;
- b. All others **depend** directly on some element;
- c. No element depends directly on **more than one** other;
and
- d. (**Non-crossing constraint**): if **A** depends directly to **B** and some element **C** intervenes between them (in linear order of string), then **C** depends directly on **A** or on **B** or to some other intervening element.





Robinson's axiom

- e. An element cannot have dependents lying on the **other side** of its own governor.(Huang, Changning)

Element:

- Head (Superior, Head, Governor, Regent, 核心词)
 - Plays the role in determining the behavior of the pair.
- Dependent (Inferior, Modifier, Subordinate, 从属词)
 - Modifier, object or complement
 - Pre-dependent, post-dependent
- Link – the relation (Arc)
 - Close to semantic relation
 - With Dependency Type

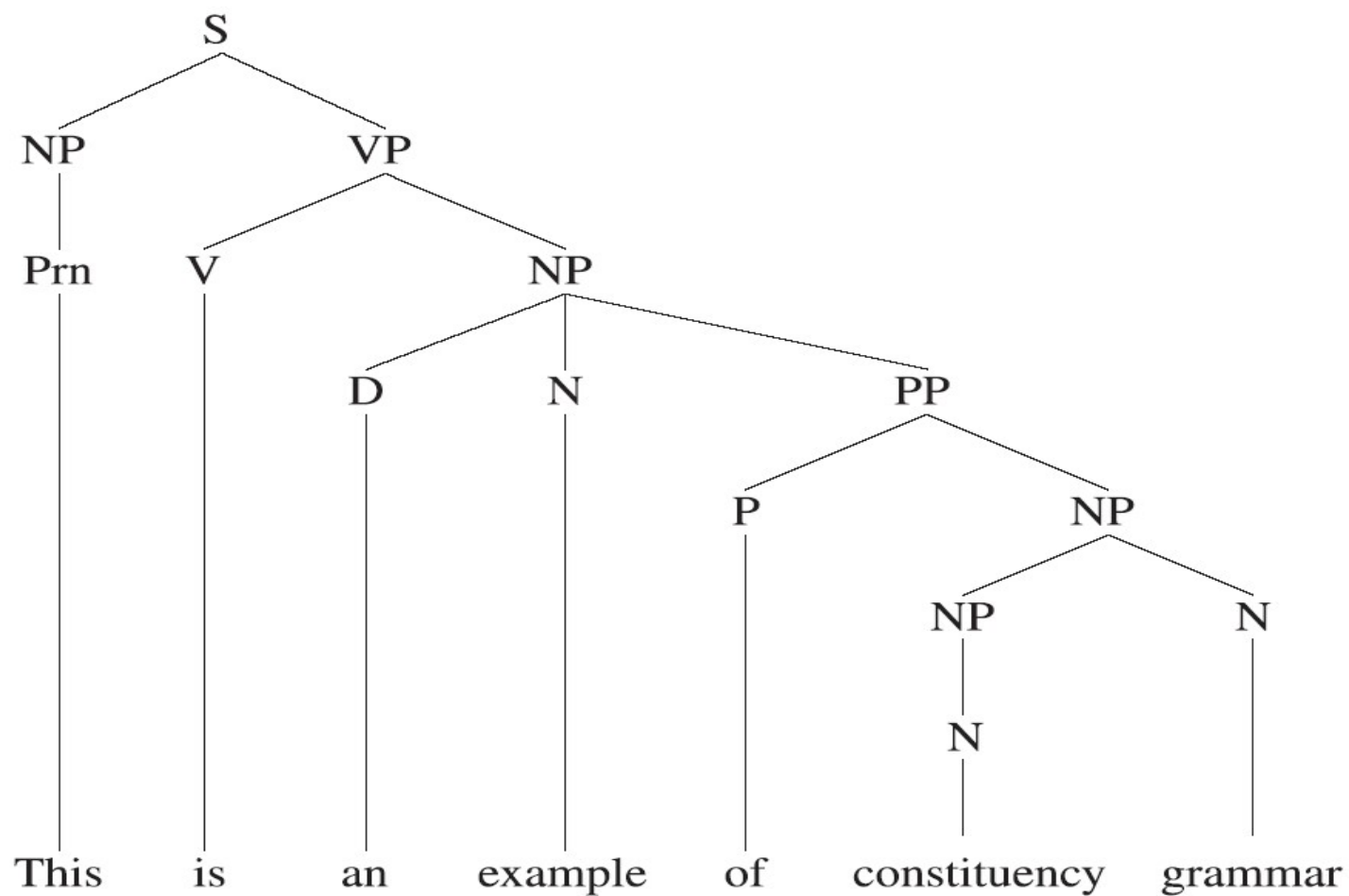


Typical Dependency Types

	Dependency Description	Example
	Adjective and its adverbial modifier	极其/d 惨痛/a <i>greatly painful</i>
<i>ADV</i>	Predicate and its adverbial modifier in which the predicate serves as head	沉重/ad 打击/v <i>heavily strike</i>
<i>AN</i>	Noun and its adjective modifier	合 法 /a 收 入 /n <i>lawful incoming</i>
<i>CMP</i>	Predicate and its complement in which the predicate serves as head	医 治 /v 无 效 /v <i>ineffectively treat</i>
<i>NJX</i>	Juxtaposition structure	公 正 /a 合 理 /a <i>fair and reasonable</i>
<i>NN</i>	Noun and its nominal modifier	人 身 /n 安 全 /n <i>personal safety</i>
<i>SBV</i>	Predicate and its subject	财 产 /n 转 移 /v <i>property transfer</i>
<i>VO</i>	Predicate and its object in which the predicate serves as head	转 换 /v 机 制 /n <i>change mechanism</i>
<i>VV</i>	Serial verb constructions which indicates that there are serial actions	跟 踪 /v 报 导 /v <i>trace and report</i>
<i>OT</i>	Others	

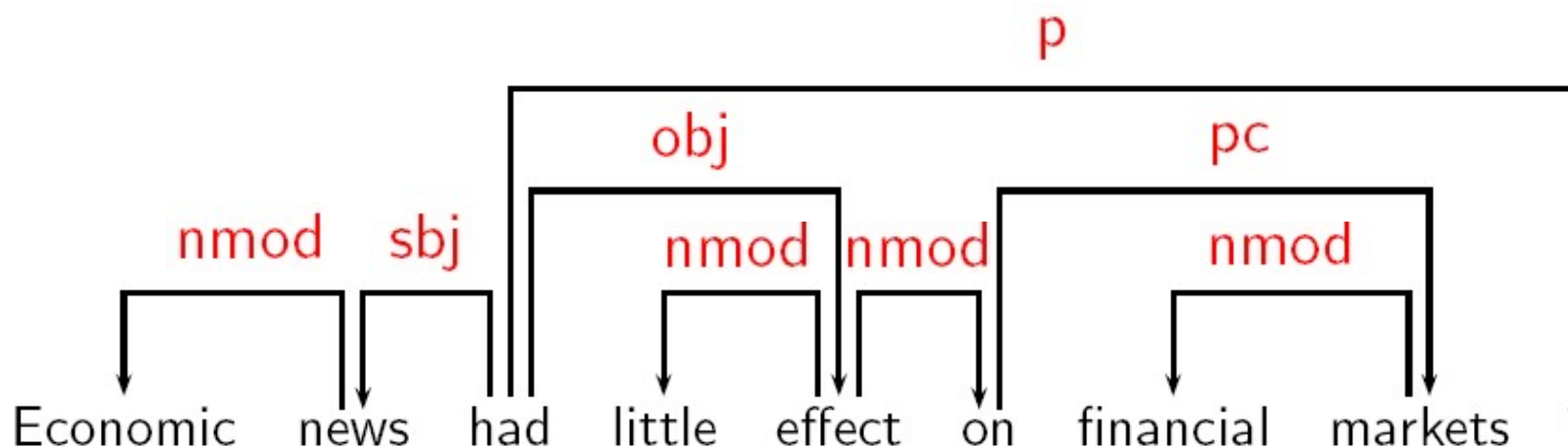
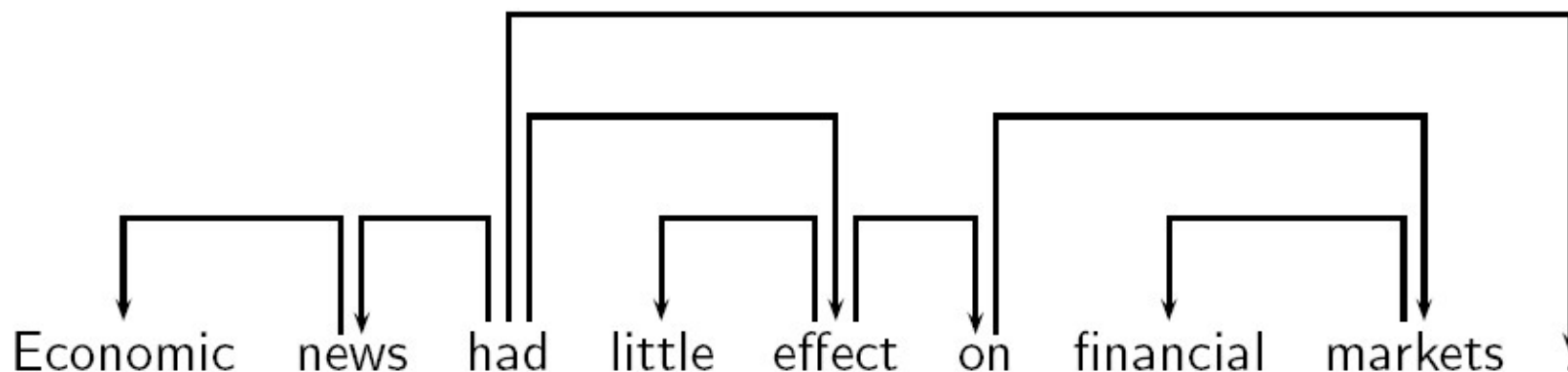


Phrase Structure





Dependency Structure





The Comparison

	Phrase structure	Dependency structure
word relation	phrasal constitution	head-dependent
categories	syntactic functional	syntactic structural
new node (Y/N)	multiple nodes per word	one node per word
operation	waiting for complete phrase	word-at-a-time



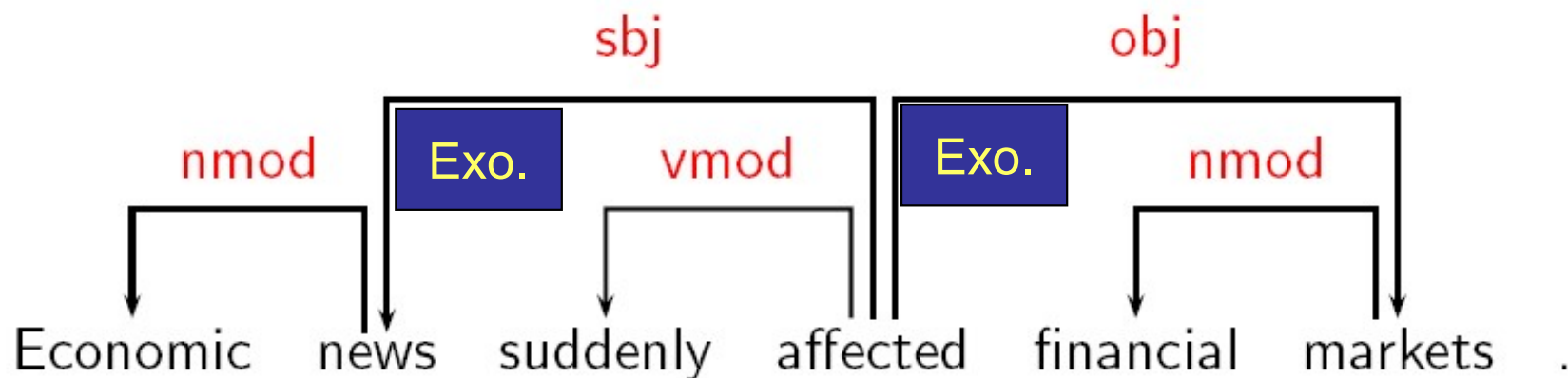
Some Theoretical Frameworks

- Word Grammar (WG) [Hudson 1984, Hudson 1990]
- Functional Generative Description (FGD) [Sgall et al. 1986]
- Dependency Unification Grammar (DUG)
 - [Hellwig 1986, Hellwig 2003]
- Meaning-Text Theory (MTT) [Melcuk 1988]
- (Weighted) Constraint Dependency Grammar ([W]CDG)
 - [Maruyama 1990, Harper and Helzerman 1995, Menzel and Schröder 1998, Schröder 2002]
- Functional Dependency Grammar (FDG)
 - [Tapanainen and Jarvinen 1997, Jarvinen and Tapanainen 1998]
- Topological/Extensible Dependency Grammar ([T/X]DG)
 - [Duchier and Debusmann 2001, Debusmann et al. 2004]



Example

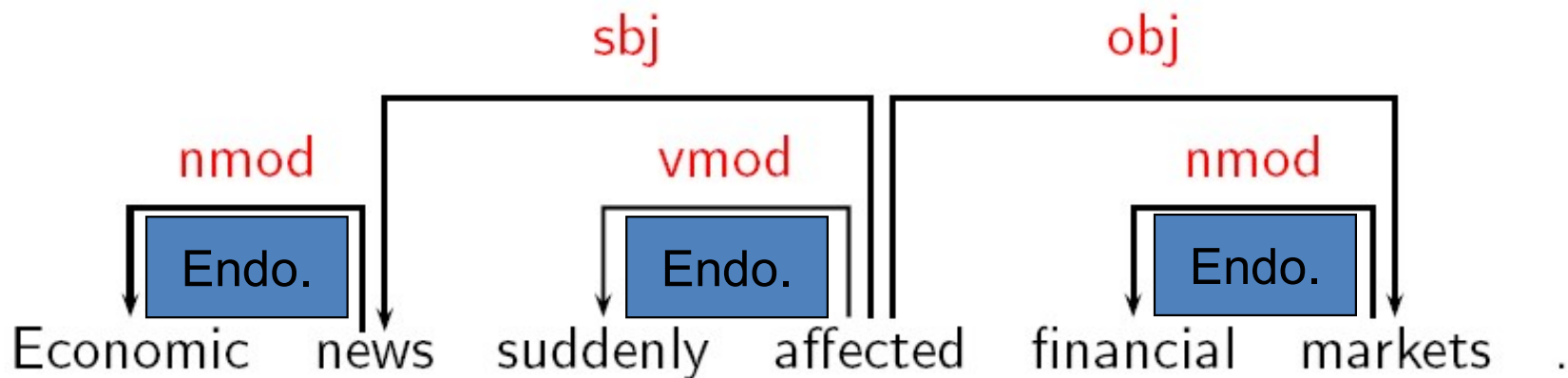
	Construction	Head	Dependent
outwards	Exocentric	Verb	Subject (sbj)
		Verb	Object (obj)
inwards	Endocentric	Verb	Adverbial (vmod)
		Noun	Attribute (nmod)





Example

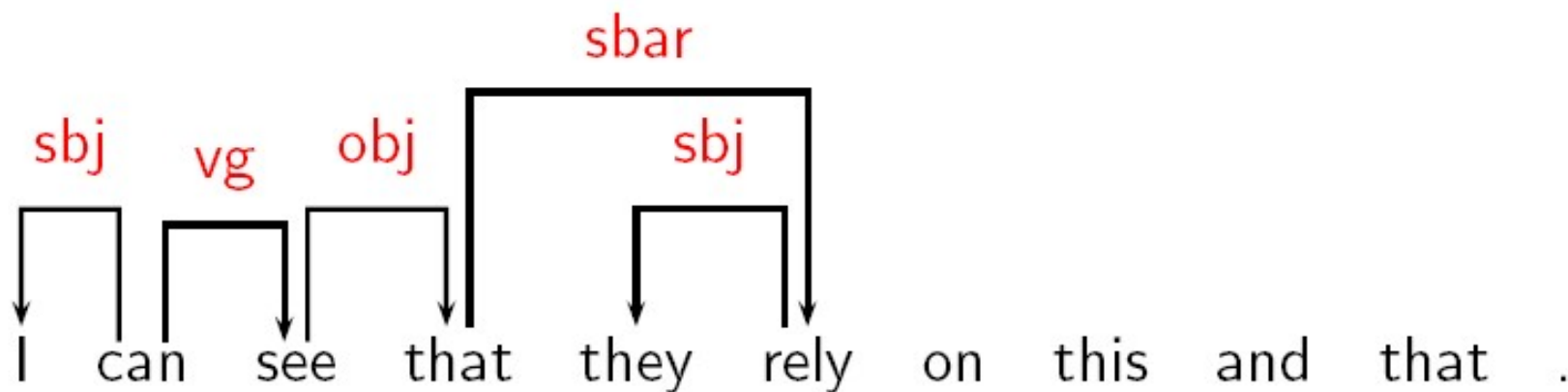
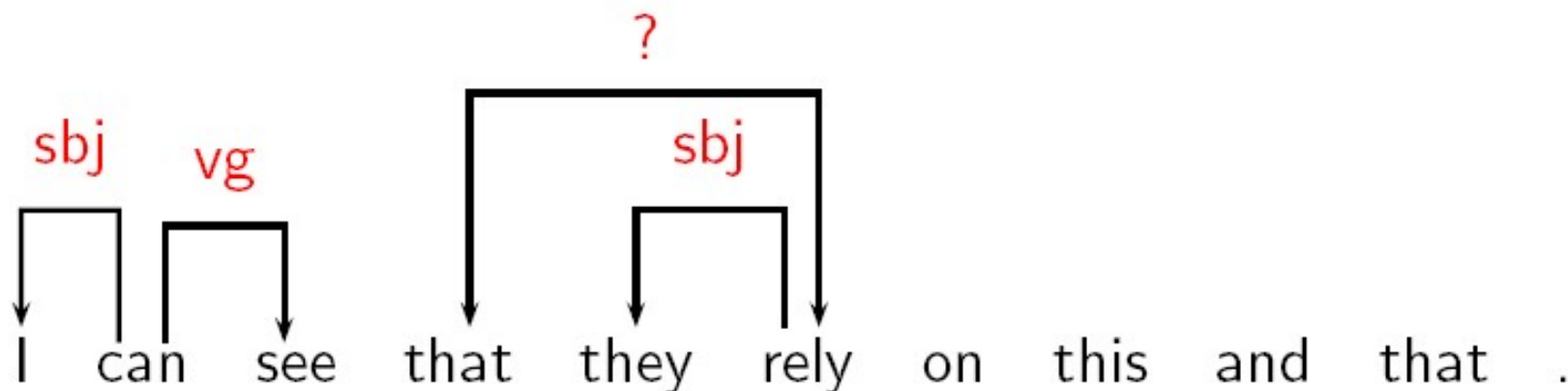
	Construction	Head	Dependent
outwards	Exocentric	Verb	Subject (sbj)
		Verb	Object (obj)
inwards	Endocentric	Verb	Adverbial (vmod)
		Noun	Attribute (nmod)





Tricky Cases

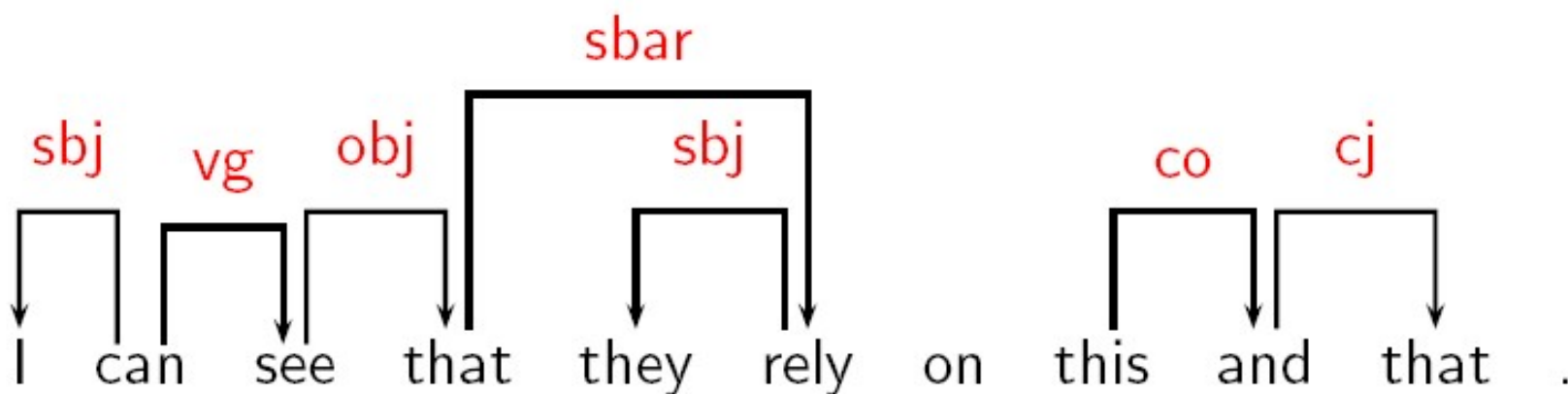
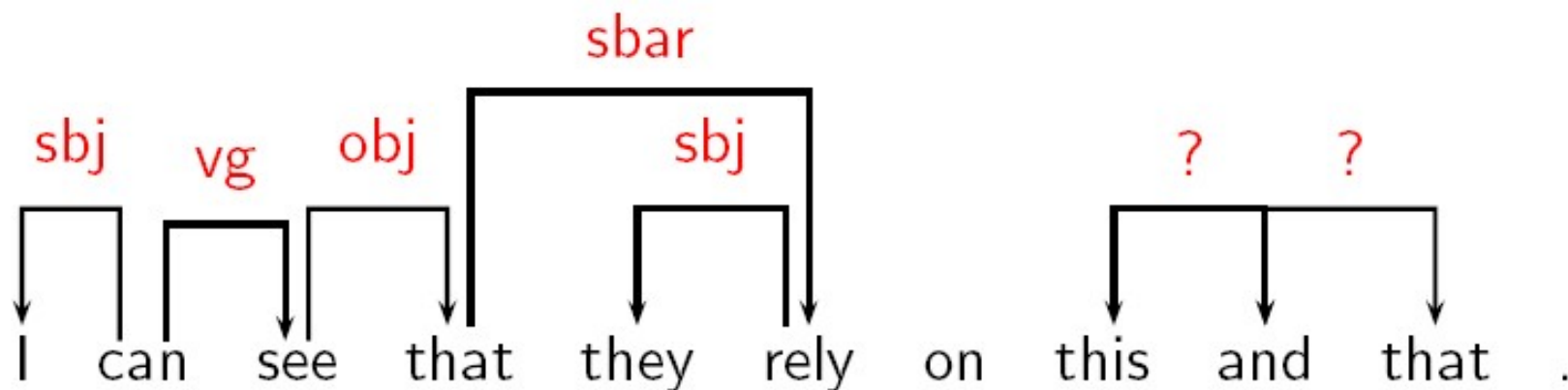
- Subordinate clauses (complementizer $\leftrightarrow$ verb)





Tricky Cases

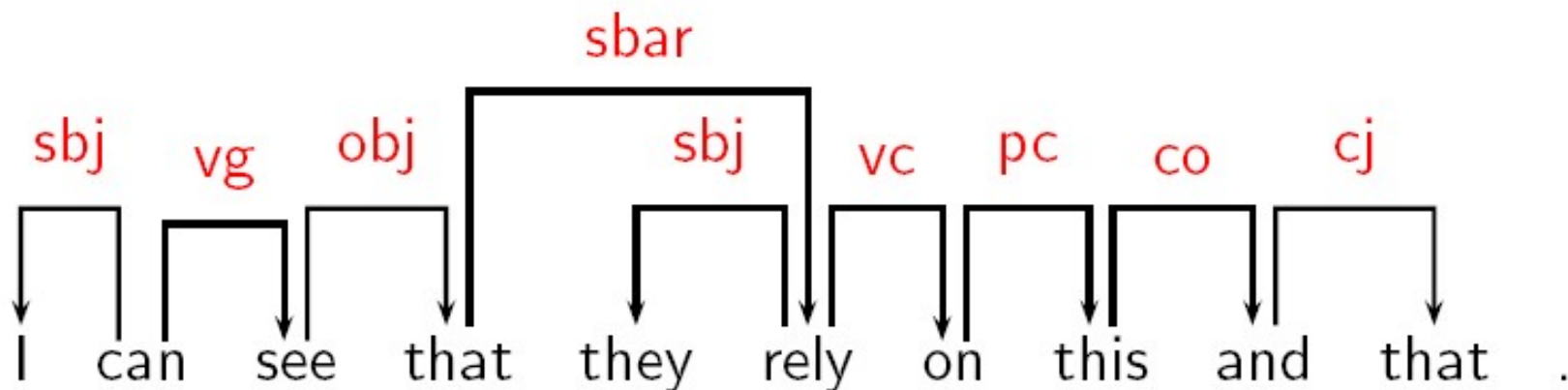
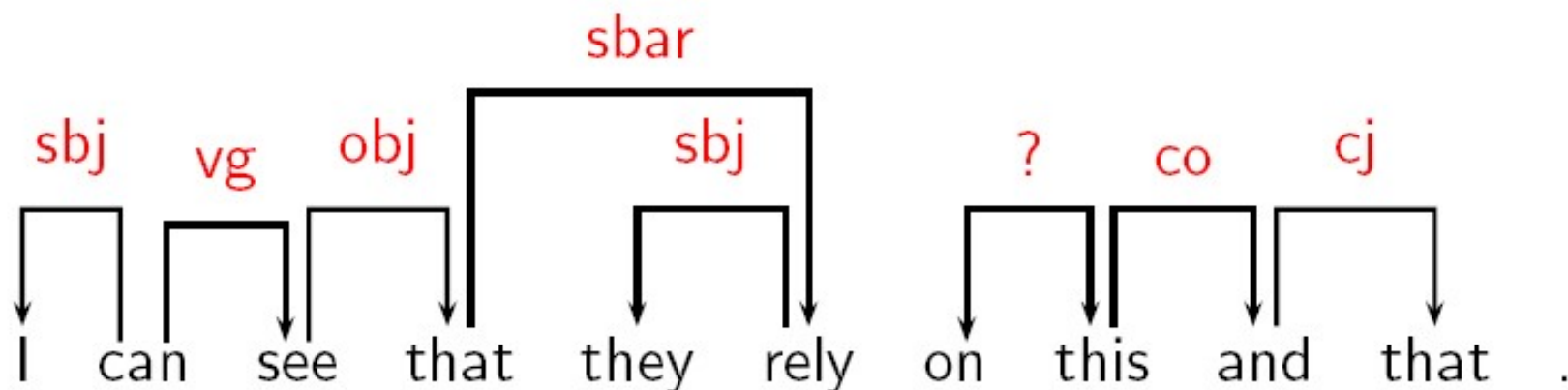
- Coordination (coordinator $\leftrightarrow$ conjuncts)





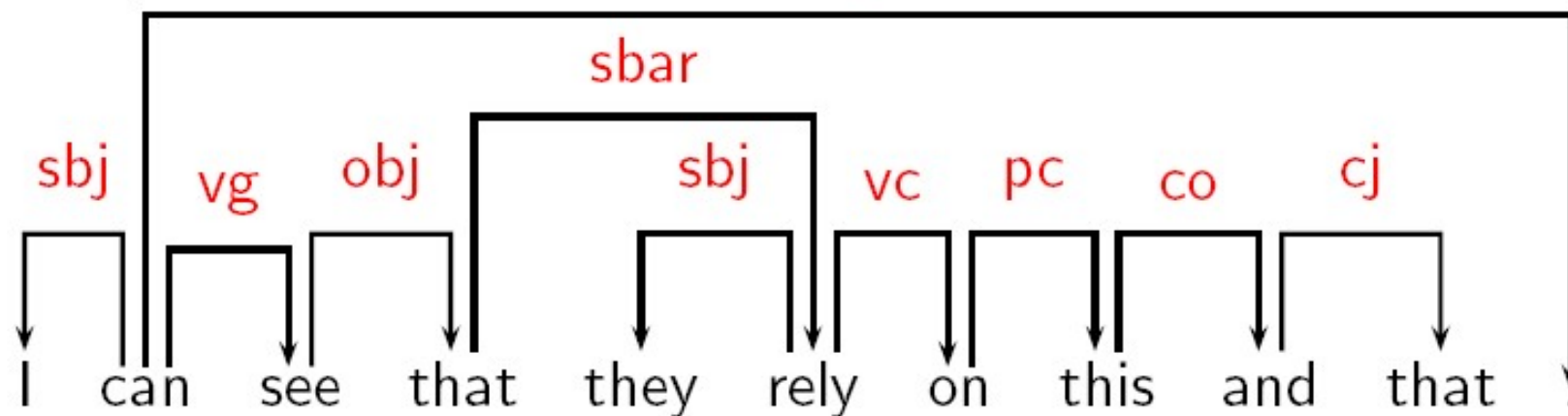
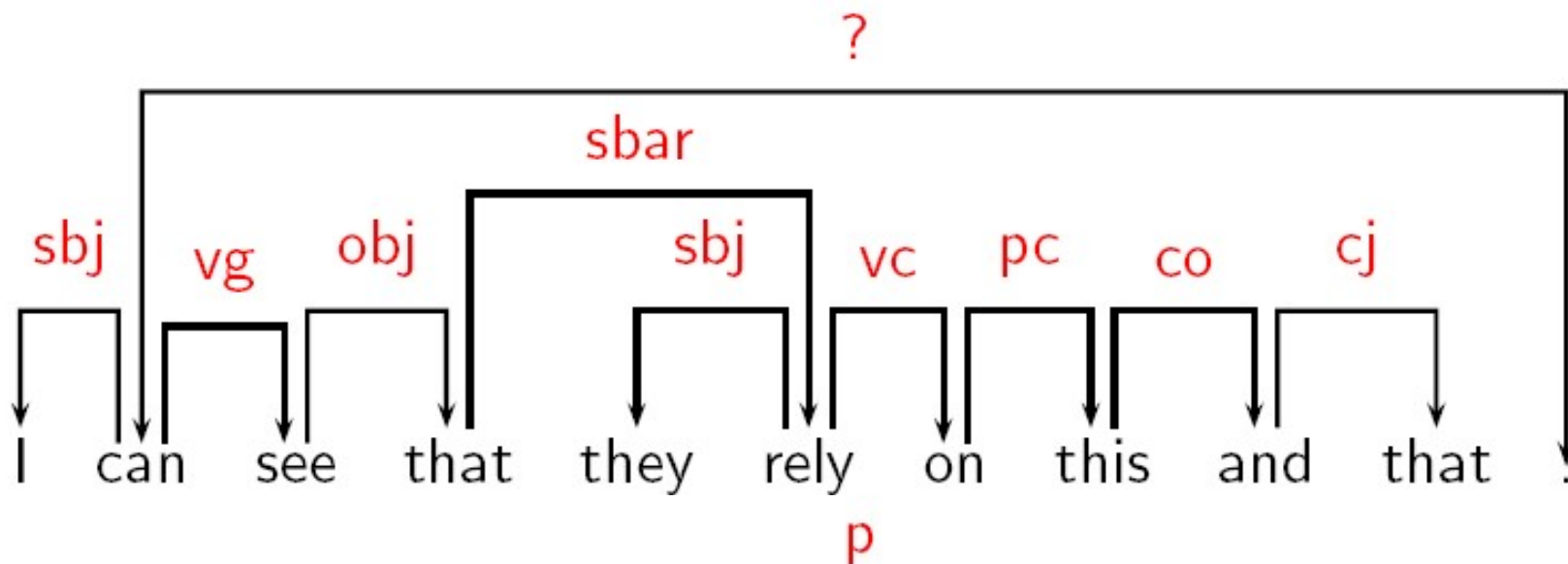
Tricky Cases

- Prepositional phrases (preposition $\leftrightarrow$ nominal)



Tricky Cases

- Punctuation





Dependency Parsing

- Input:
 - Sentence $W = w_0, w_1, \dots, w_n$ with $w_0 = \text{root}$
- Output:
 - Dependency graph $G = (V, A)$ for W where:
 - $V = \{0, 1, \dots, n\}$ is the *vertex* set,
 - A is the *arc* set, i.e., $(i, j, k) \in A$ represents a dependency from w_i to w_j with label $l_k \in L$

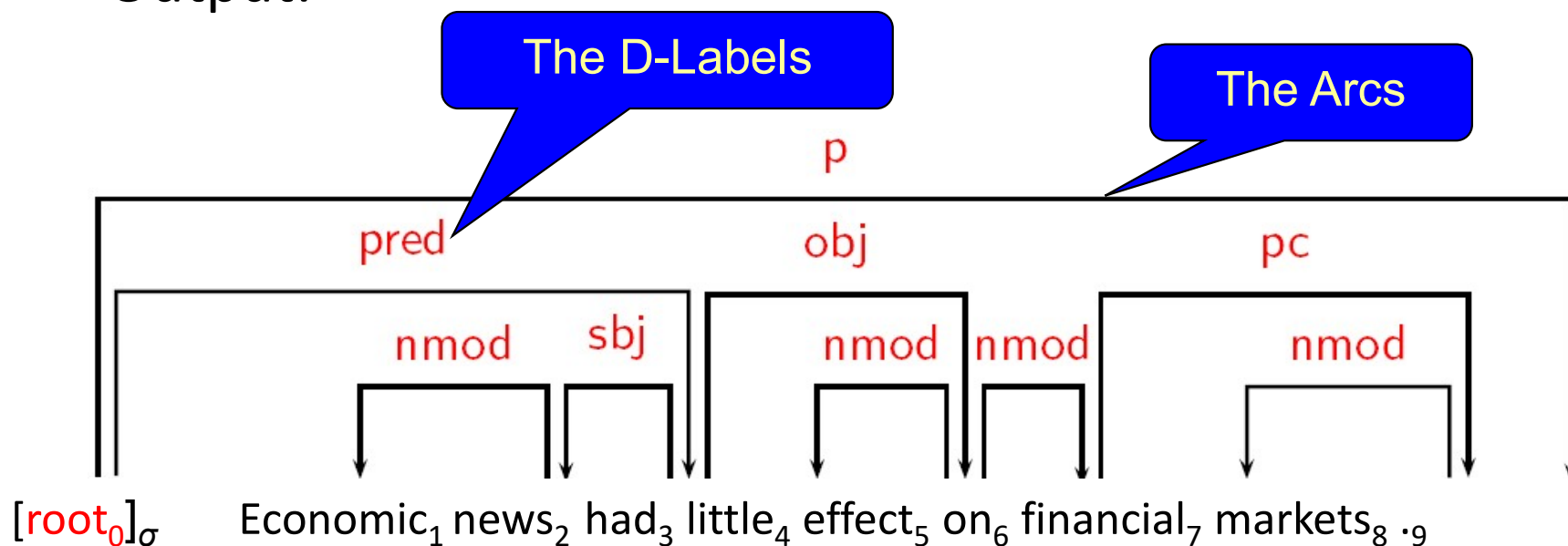


Example

- Input:

$[\text{root}_0]_\sigma$ Economic₁ news₂ had₃ little₄ effect₅ on₆ financial₇ markets₈ .₉

- Output:





Approaches

- Grammar-based parsing
 - Context-free dependency grammar
 - Constraint dependency grammar
- Data-driven parsing
 - Transition-based models
 - Graph-based models

Context-free Dependency Grammar



- Basic idea
 - Dependency grammar as **lexicalized context-free** grammar
 - Standard context-free parsing algorithms (CKY, Earley, etc.)
- Recent developments:
 - Link Grammar [Sleator and Temperley 1991]
 - Earley-style parser with left-corner filtering [Lombardo and Lesmo 1996]
 - Bilexical grammars [Eisner 1996, Eisner 2000]

Constraint Dependency Grammar



- Parsing as **constraint satisfaction** [Maruyama 1990]:
 - Grammar consists of a set of boolean constraints, i.e. logical formulas that describe well-formed dependency graphs.
 - Constraint propagation removes candidate graphs that contradict constraints (eliminative parsing).
- Recent developments:
 - Weighted Constraint Dependency Grammar [Menzel and Schröder 1998, Foth et al. 2004]
 - Probabilistic Constraint Dependency Grammar [Harper and Helzerman 1995, Wang and Harper 2004]
 - Topological/Extensible Dependency Grammar [Duchier and Debusmann 2001, Debusmann et al. 2004]



Transition-based models

- Basic idea:
 - Define a **transition** system (state machine, MM) for mapping a sentence to its dependency graph.
 - **Learning**: Induce a model for predicting the next state transition, given the transition history.
 - **Parsing**: Construct the optimal transition sequence, given the induced model.
- Characteristics:
 - Local training of a model for optimal transitions
 - Greedy search/inference



Transition Systems

- A **transition system** for dependency parsing is a quadruple $S = (C, T, c_s, C_t)$, where
 1. C is a set of **configurations**, each of which contains a buffer β of (remaining) nodes and a set A of dependency arcs,
 2. T is a set of **transitions**, each of which is a (partial) function $t: C \rightarrow C$,
 3. c_s is an **initialization** function, mapping a sentence $x = W_0, W_1, \dots, W_n$ to a configuration with $\beta = [1, \dots, n]$,
 4. $C_t \subseteq C$ is a set of **terminal** configurations.
- Note:
 - A **configuration** represents a **parser state**.
 - A **transition** represents a **parsing action** (parser state update).



Transition Sequences

- Let $S = (C, T, c_s, C_t)$ be a transition system.
- A **transition sequence** for a sentence $x = W_0, W_1, \dots, W_n$ in S is a sequence $C_{0,m} = (C_0, C_1, \dots, C_m)$ of configurations, such that
 1. $c_0 = c_s(x)$,
 2. $C_m \in C_t$,
 3. For every i ($1 \leq i \leq m$), $C_i = t(c_{i-1})$ for some $t \in T$.
- The parse assigned to x by $C_{0,m}$ is the dependency graph $G_{Cm} = (\{0, 1, \dots, n\}, A_{Cm})$, where A_{Cm} is the set of dependency arcs in C_m .

Lexicalized Dependency Probabilistic Model



- Collins, 1996
- Training: Retrieve the probability of any words with specific dependency through Maximal Likelihood Estimation.
- Parsing (Decoding): Search a dependency tree with the highest product of production probability of all dependent pairs

Probabilistic Dependency Model



- Eisner, 1996
- For each candidate dependency tree, T , its production probability is defined as the probability product of the production probability of each node in T

$$Gen(T) = \prod_{x \in T} Gen(x)$$

- Task of Parsing: Looking for the dependency tree with the highest production probability

Shift-Reduce Dependency Parsing



- Transitions:
 - **Left-Arc_k:**
 $(\sigma | i, j | \beta, A) \rightarrow (\sigma, j | \beta, A \cup \{(j, i, k)\})$
 - **Right-Arc_k:**
 $(\sigma | i, j | \beta, A) \rightarrow (\sigma, i | \beta, A \cup \{(i, j, k)\})$
 - **Shift:**
 $(\sigma, i | \beta, A) = (\sigma | i, \beta, A)$
- Preconditions:
 - **Left-Arc_k:**
 $\neg[i = 0]$
 $\neg \exists i' \exists k' [(i', i, k') \in A]$
 - **Right-Arc_k:**
 $\neg \exists i' \exists k' [(i', j, k') \in A]$

Example: Shift-Reduce Parsing - 1



$[\text{root}_0]_\sigma [\text{Economic}_1 \text{ news}_2 \text{ had}_3 \text{ little}_4 \text{ effect}_5 \text{ on}_6 \text{ financial}_7 \text{ markets}_8 \text{ .}_9]_\beta$

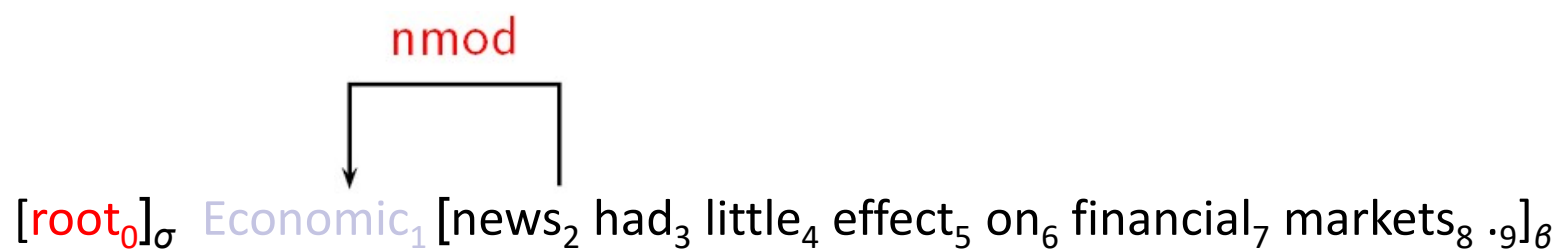
We have nothing in the configuration.
Just shift to first word



[root₀ Economic₁]_σ [news₂ had₃ little₄ effect₅ on₆ financial₇ markets₈ .₉]_β

Shift

We have one word in the configuration. Now
to search the in/out arcs ...



Left-Arc_{nmod}

A left-arc is found. Include the dependent in current configuration ...

nmod

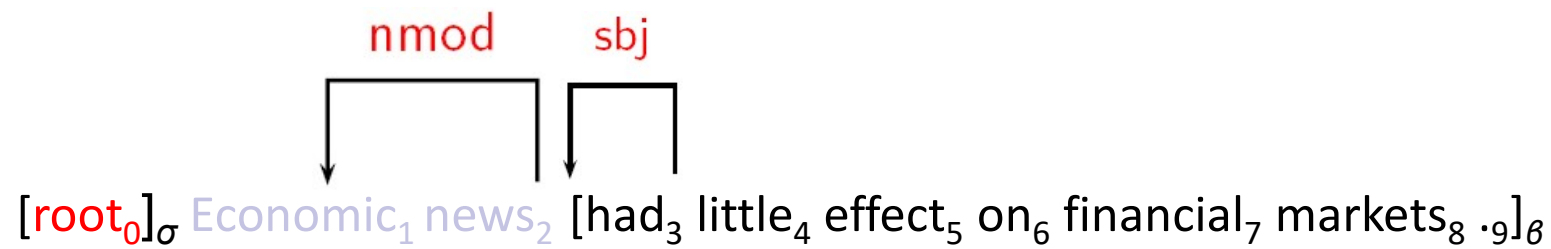
└──┘

↓

[root₀ Economic₁ news₂]_σ [had₃ little₄ effect₅ on₆ financial₇ markets₈ .₉]_θ

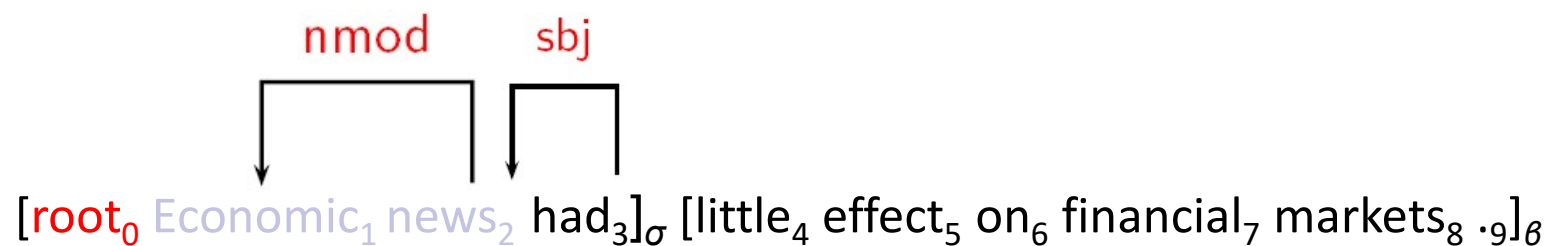
Shift

Current configuration is self-contained. Now shift is required to include the next word ...



Left-Arc_{sbj}

But the next word holds a *SBJ* left arc to word *news*.
The configuration is reduced to empty again.



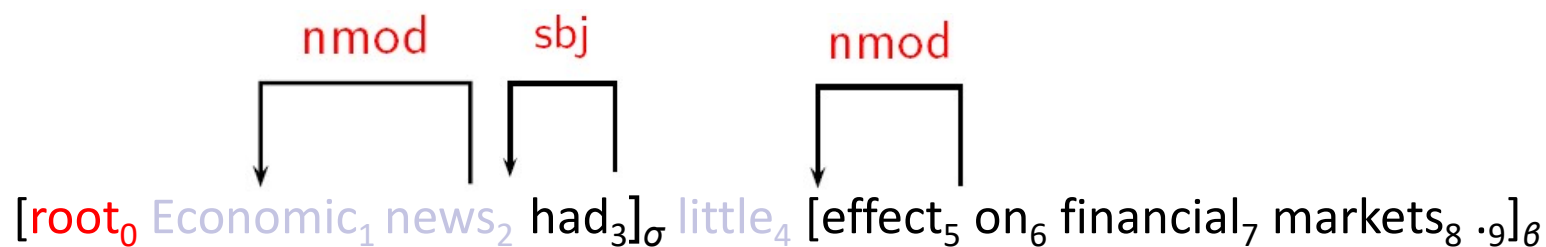
Shift

The *SBJ* left arc creates a new configuration. We now shift to the next word.



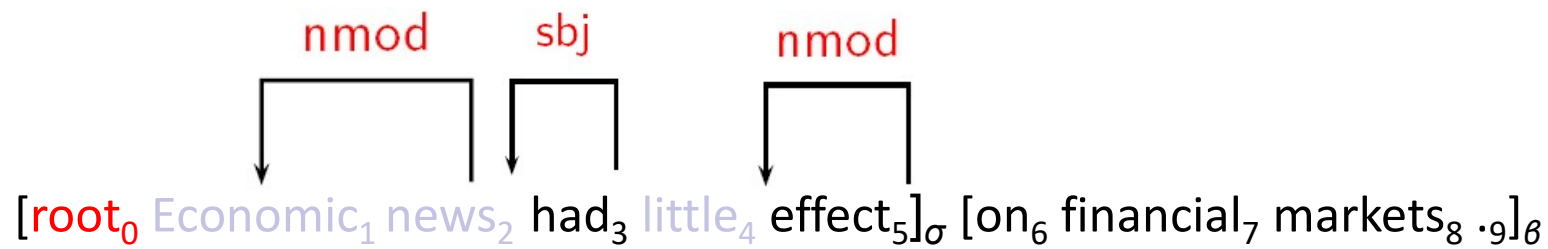
Shift

The next word holds no valid arc with current configuration. Then we continue to shift.



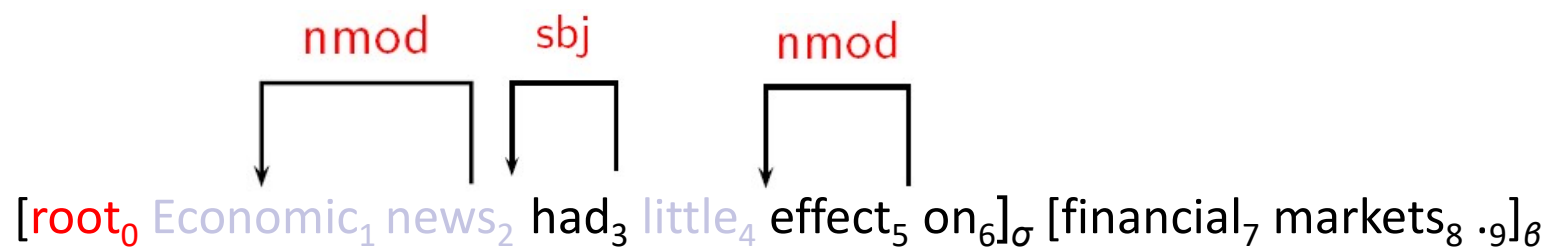
Left-Arc_{nmod}

Now a NMOD left arc is found. The current configuration is reduced to previous one.



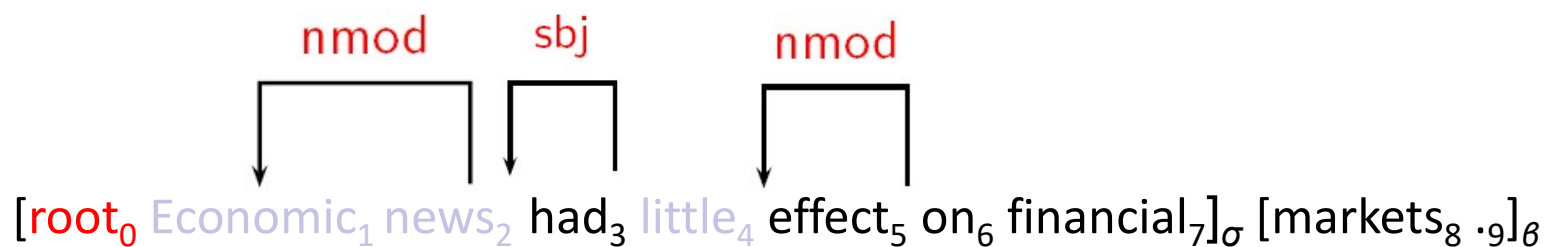
Shift

The valid NMOD left arc is included in the current configuration. Shift to next word.



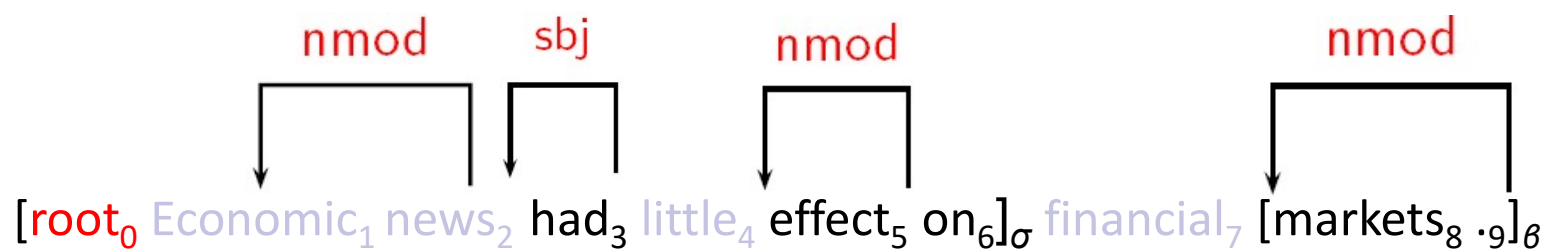
Shift

The word on should hold an arc to its neighbors. It is not word effect. So we continue to shift.



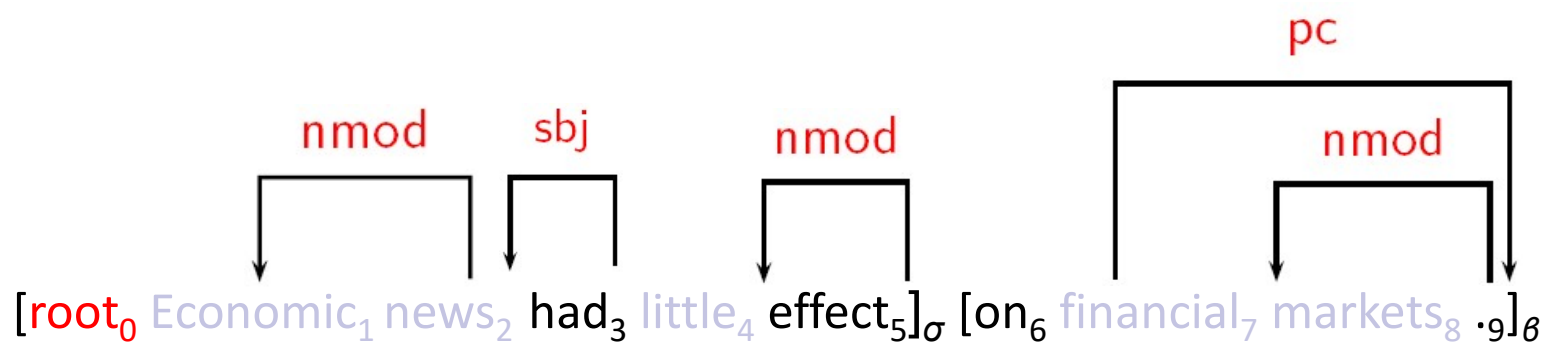
Shift

We include the new word in the configuration. However, the arc from *on* to *financial* is not valid. Shift but reduce to previous configuration.



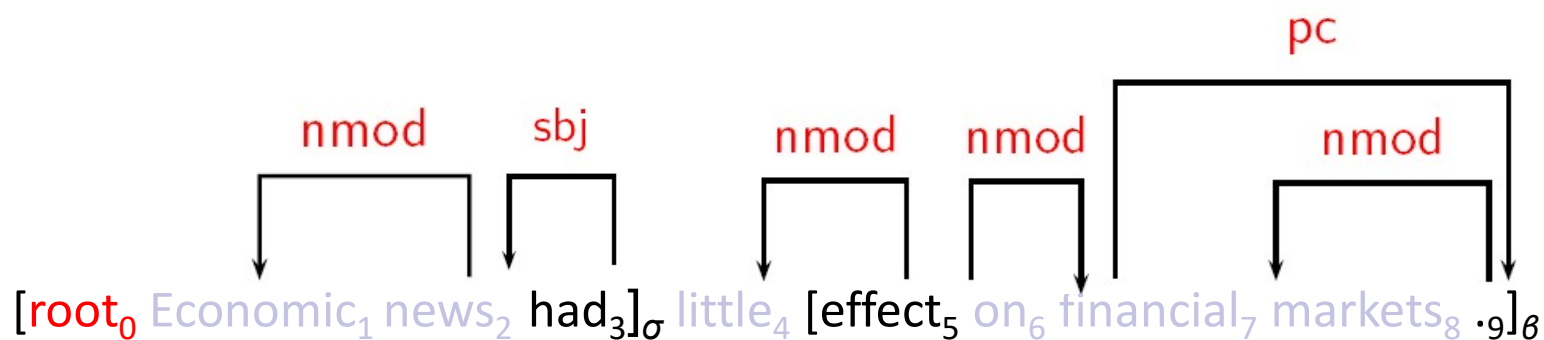
Left-Arc_{nmod}

New NMOD left arc is found. The remaining word is punctuation. Stop shifting. Search right arc.



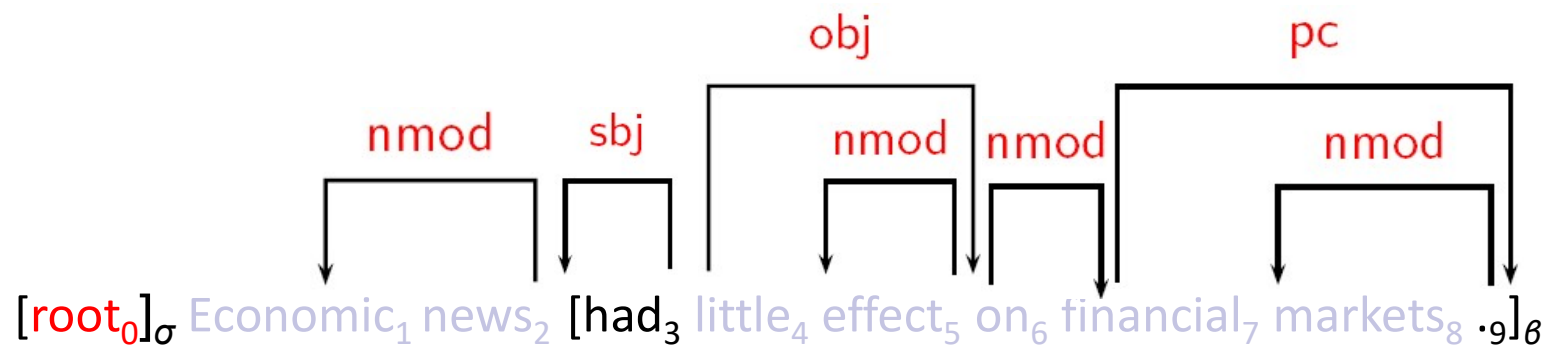
Right-Arc_{pc}

A PC right arc is found from *on* to *markets*. Reduce configuration. Continue searching right arc.



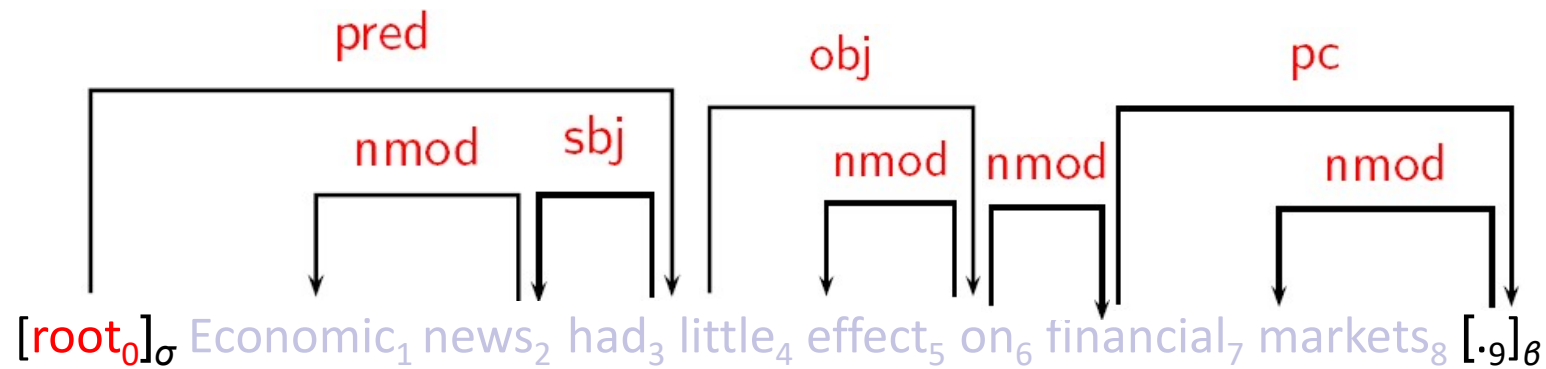
Right-Arc_{nmod}

A PC right arc is found from *effect* to *on*. Reduce configuration. Continue searching right arc.



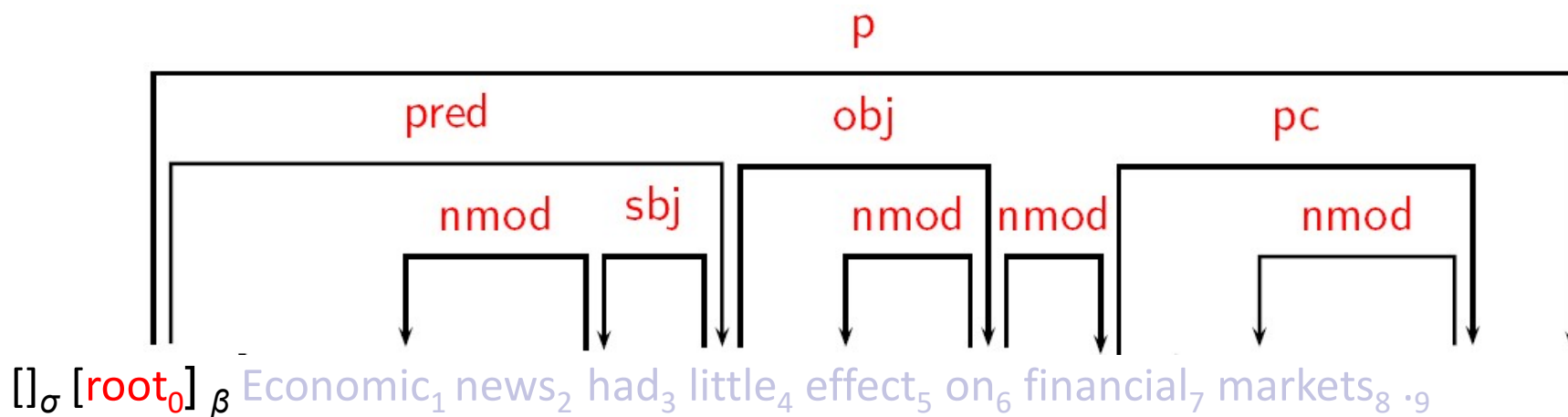
Right-Arc_{obj}

A PC right arc is found from *had* to *effect*. Reduce configuration. Continue searching right arc.



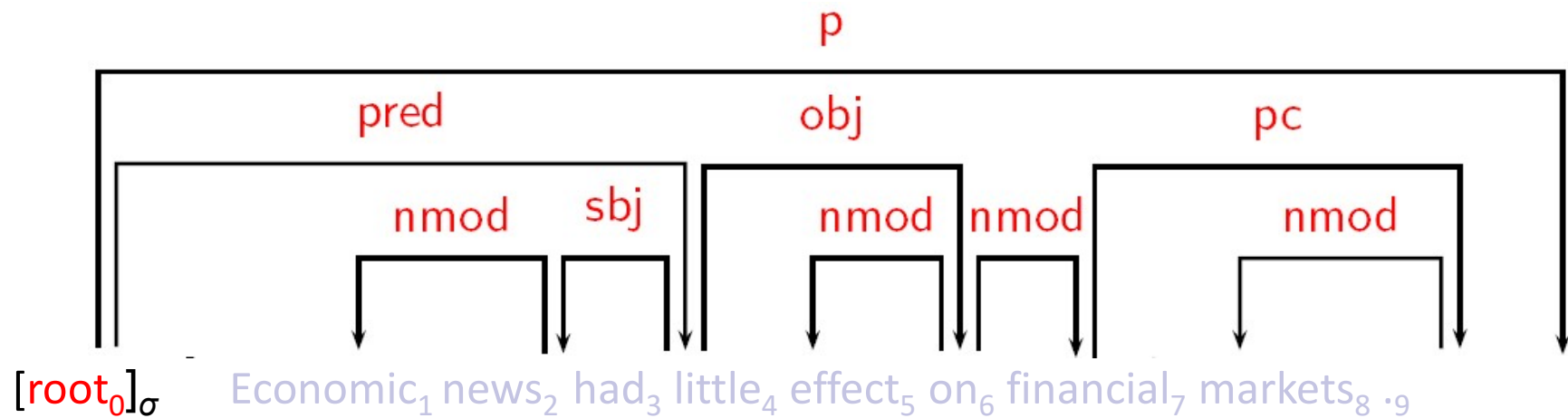
Right-Arc_{pred}

A PC right arc is found from *root* to *had*. Reduce configuration. Continue searching right arc.



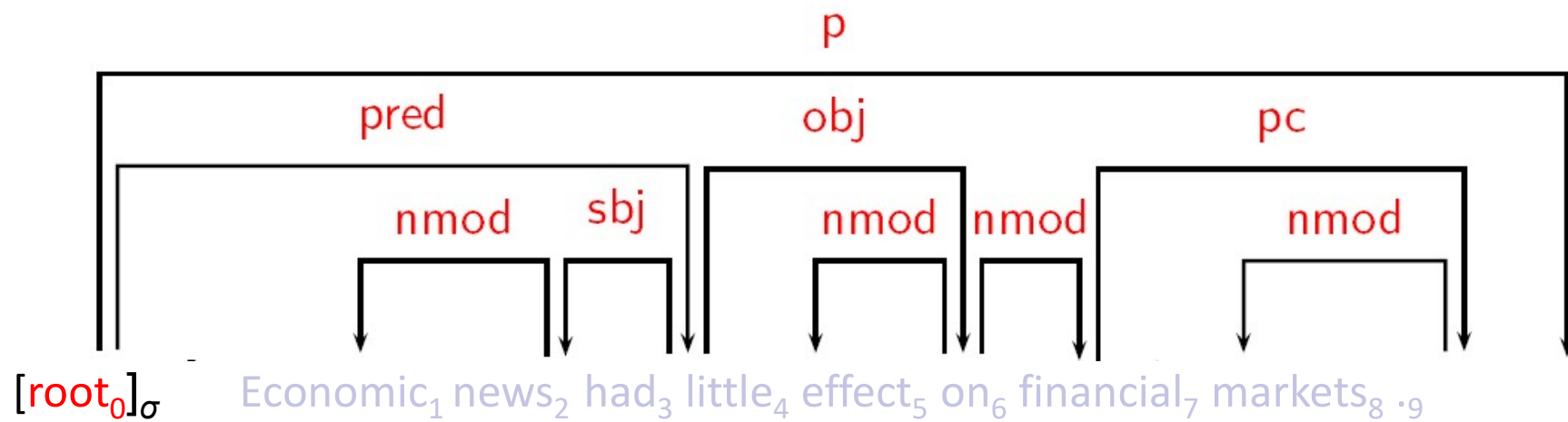
Right-Arc_p

A PC right arc is found from *root* to *punctuation*. Reduce configuration. Continue searching right arc.



Shift

No right arc is found. Remove the empty configuration.
Parsing over.



Shift

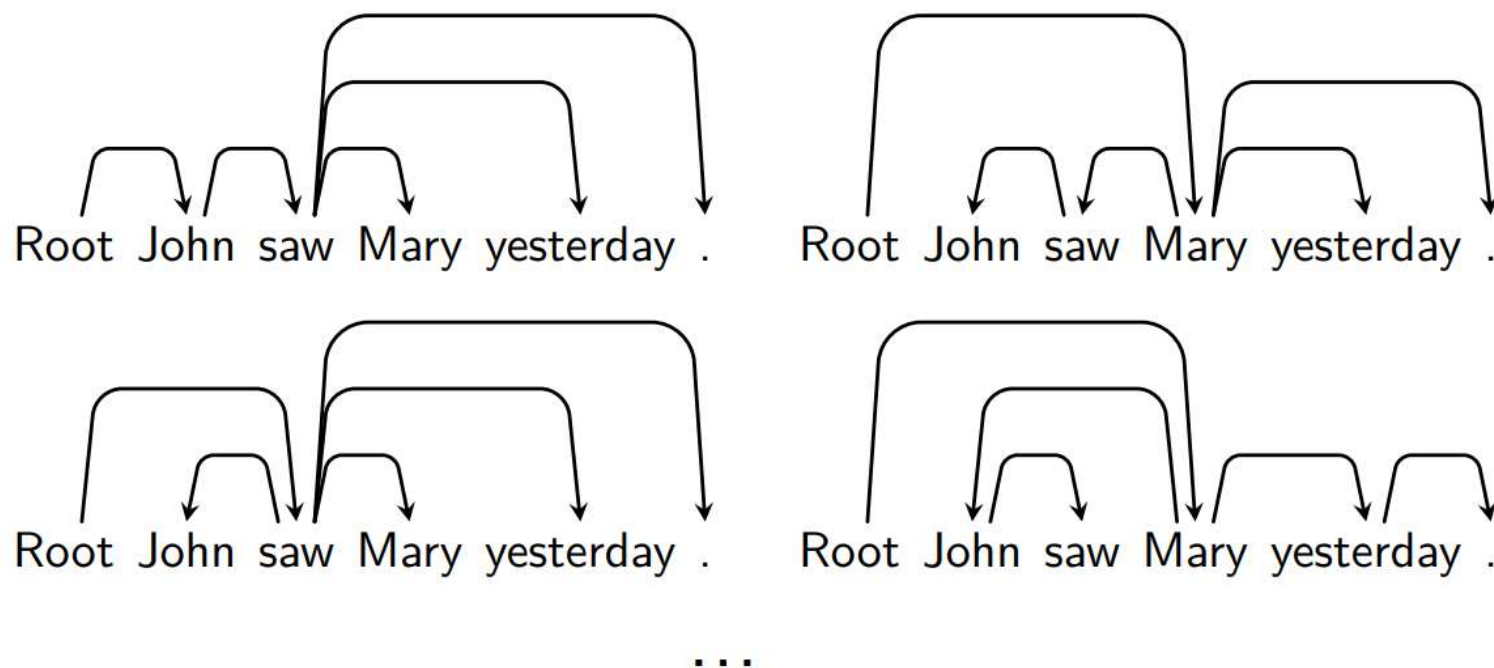


Graph-based Models

- Basic idea:
 - Define a space of candidate **dependency graphs** for a sentence.
 - **Learning**: Induce a model for **scoring** an entire dependency graph for a sentence.
 - **Parsing**: Find the highest-scoring dependency graph, given the induced model.
- Characteristics:
 - Global training of a model for optimal dependency graphs
 - Exhaustive search/inference

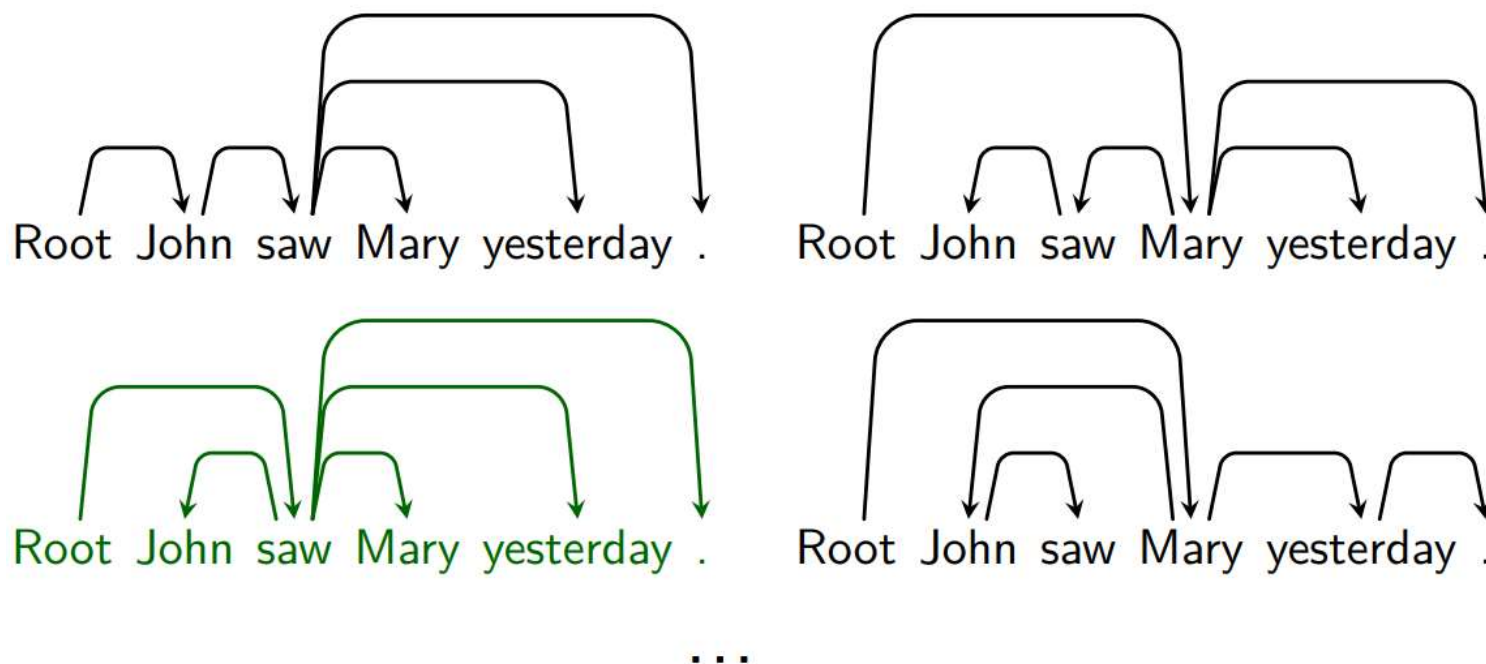
Graph-based Models

- Candidate trees for “John saw mary yesterday”



Graph-based Models

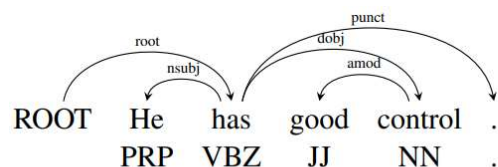
- Candidate trees for “John saw mary yesterday”





Neural-based Dependency Parsing

- Chen, Danqi, and Christopher D. Manning, EMNLP 2014
- VS. traditional transition-based approach
- Let the neural network do the feature combination



Correct transition: SHIFT



Transition	Stack	Buffer	A
	[ROOT]	[He has good control .]	$\emptyset$
SHIFT	[ROOT He]	[has good control .]	
SHIFT	[ROOT He has]	[good control .]	
LEFT-ARC (nsubj)	[ROOT has]	[good control .]	$A \cup \text{nsubj}(\text{has}, \text{He})$
SHIFT	[ROOT has good]	[control .]	
SHIFT	[ROOT has good control]	[.]	
LEFT-ARC (amod)	[ROOT has control]	[.]	$A \cup \text{amod}(\text{control}, \text{good})$
RIGHT-ARC (dobj)	[ROOT has]	[.]	$A \cup \text{dobj}(\text{has}, \text{control})$
...	...	...	...
RIGHT-ARC (root)	[ROOT]	[]	$A \cup \text{root}(\text{ROOT}, \text{has})$

Single-word features (9)

$s_1.w; s_1.t; s_1.wt; s_2.w; s_2.t;$
 $s_2.wt; b_1.w; b_1.t; b_1.wt$

Word-pair features (8)

$s_1.wt \circ s_2.wt; s_1.wt \circ s_2.w; s_1.wts_2.t;$
 $s_1.w \circ s_2.wt; s_1.t \circ s_2.wt; s_1.w \circ s_2.w$
 $s_1.t \circ s_2.t; s_1.t \circ b_1.t$

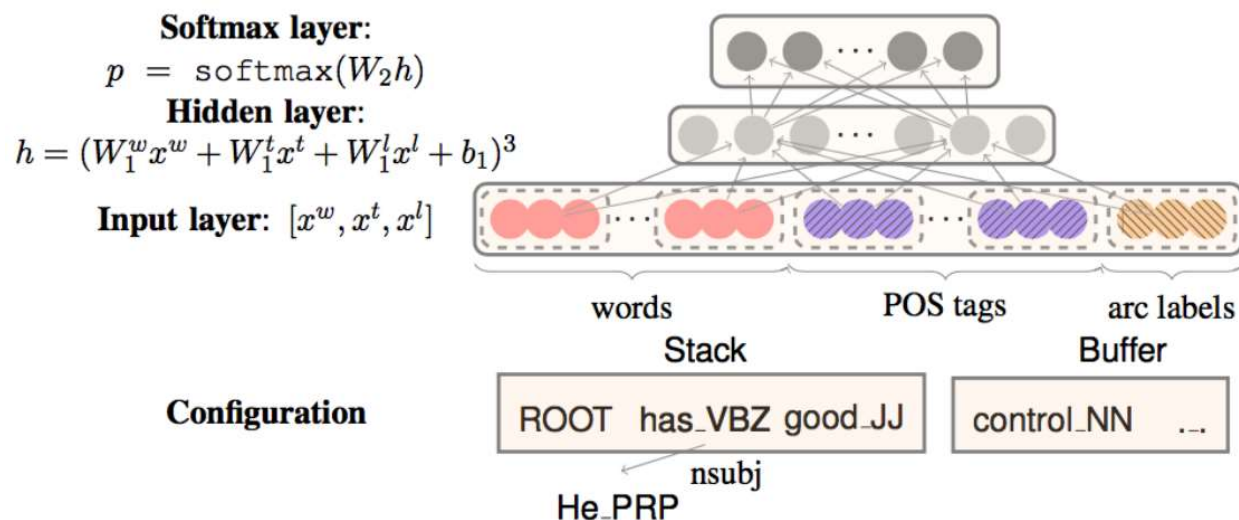
Three-word faeures (8)

$s_2.t \circ s_1.t \circ b_1.t; s_2.t \circ s_1.t \circ lc_1(s_1).t;$
 $s_2.t \circ s_1.t \circ rc_1(s_1).t; s_2.t \circ s_1.t \circ lc_1(s_2).t;$
 $s_2.t \circ s_1.t \circ rc_1(s_2).t; s_2.t \circ s_1.w \circ rc_1(s_2).t;$
 $s_2.t \circ s_1.w \circ lc_1(s_1).t; s_2.t \circ s_1.w \circ b_1.t$



Neural-based Dependency Parsing

- The Input layer
 - Word embedding matrix E_w , POS tags embedding matrix E_t , Arc label embedding matrix E_l
 - Choose a set of elements based on the stack / buffer positions for each type of information (word, POS or label)
 - Add word feature $x_w = [e_{w_1}^w, e_{w_1}^w, \dots, e_{w_n}^w]$ to the input layer, Similarly, we add the POS tag features x_t and arc label features x_l to the input layer.





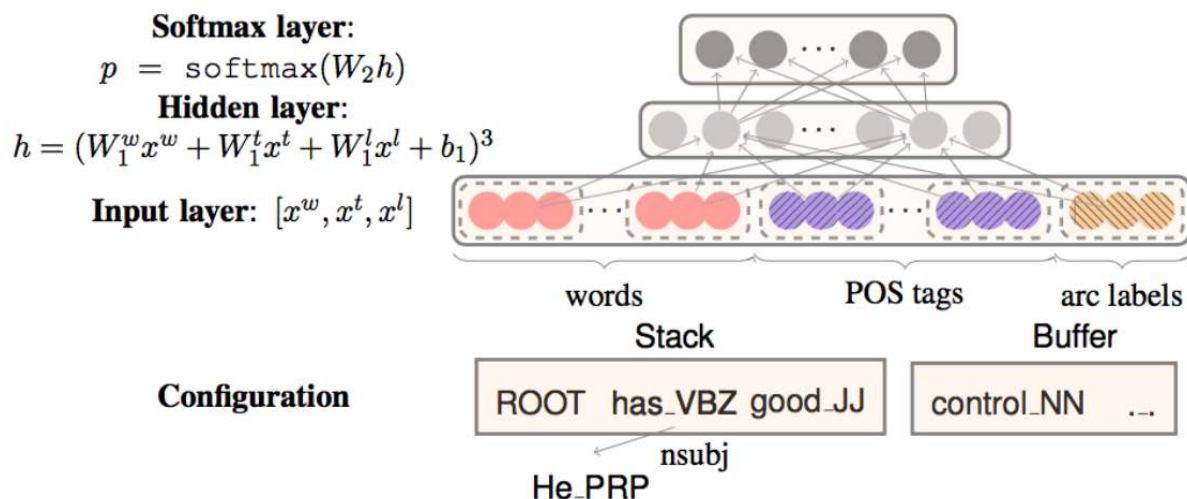
Neural-based Dependency Parsing

- The neural network architecture with one hidden layer
 - map the input layer to a hidden through a cube activation function:

$$h = (W_1^w x_w + W_1^t x_t + W_1^l x_l + b_1)^3$$

- A softmax layer is finally added on the top of the hidden layer for modeling multi-class probabilities

$$p = \text{softmax}(W_2 h)$$



Neural-based Dependency Parsing



- What Features to Extract?
 - The Top3 words on the stack and buffer (6 features)
 - The two leftmost/rightmost children of the top tow words on the stack (8 features)
 - Leftmost and rightmost grandchildren (4 features)
 - POS tags of all the above (18 features)
 - Arc labels of all children/grandchildren (12 features)

Neural-based Dependency Parsing



- Contributions
 - The first simple, successful neural dependency parser
 - The dense representations let it outperform other greedy parsers in both accuracy and speed
 - This work was further developed and improved by others, particular at Google
 - Bigger , deeper networks with better tuned hyperparameters
 - Beam search
 - Global, conditional random field (CRF)-style inference over the decision sequence



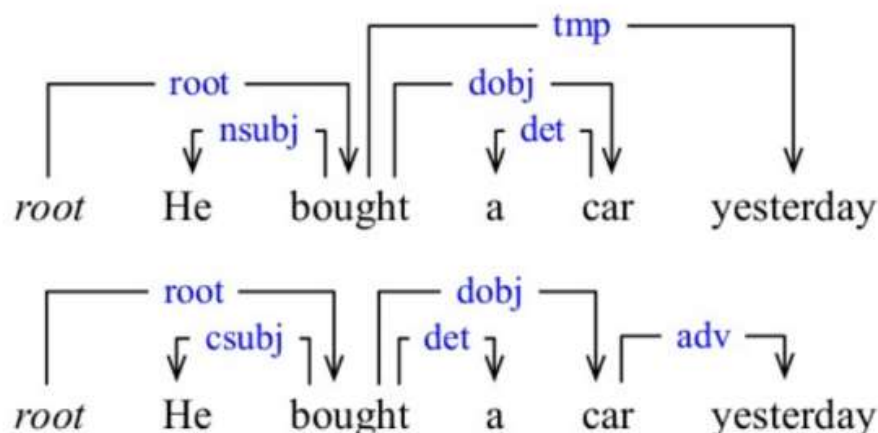
Parsing Evaluation

- Assume each node has exactly one head except for the ***root***
- For each tree, count
 - how many nodes found correct heads:
 - Unlabeled attachment score (UAS)
 - how many nodes found correct labels:
 - Label accuracy (LS)
 - how many nodes found both correct heads and labels:
 - Labeled (LAS)



Parsing Evaluation

- Unlabeled attachment score
 - Mismatches: bought -> a , car -> yesterday (3 / 5 = 60%)
- Label accuracy
 - Mismatches: He - csubj , yesterday - adv (3 / 5 = 60%)
- Labeled attachment score
 - He - csubj , bought -> a , car -> yesterday - adv (2 / 5 = 60%)





Treebanks

- **English Penn Treebank**: Standard corpus for testing syntactic parsing consists of 1.2 M words of text from the Wall Street Journal (WSJ).
- Typical to train on about 40,000 parsed sentences and test on an additional standard disjoint test set of 2,416 sentences.
- **Chinese Penn Treebank**: 100K words from the Xinhua news service.
- Other corpora existing in many languages, see the Wikipedia article “Treebank”



Parsing Resources

- Michael Collins 's Parser : English
 - <http://people.csail.mit.edu/mcollins/code.html>
- Dan Bikel 's Parser: English / Chinese / Arabic
 - <http://www.cis.upenn.edu/~dbikel/software.html#stat-parser>
- Stanford Parser: English / Chinese / German
 - <http://www-nlp.stanford.edu/software/lex-parser.shtml>
 - English / Chinese / German
- David Chiang's Parser: English / Chinese
 - <http://www.isi.edu/~chiang/>
- HIT IR Dependency Parser
 - <http://ir.hit.edu.cn>



The Next Lecture

- Lecture 10
Text Categorization